

INGENIERÍA DE SOFTWARE I

BIENVENIDA

Bienvenido(a) a la asignatura *Ingeniería de Software I,* con la cual comprenderás los principios básicos del proceso de desarrollo de software. Algunos de estos son: los tipos de sistemas de información, los modelos de desarrollo de software, la gestión de los procesos de desarrollo y las métricas necesarias para desarrollar un producto de calidad.

Este curso será la base para que puedas contestar preguntas como: ¿Qué recursos se requieren para desarrollar un software? ¿Cómo se pueden optimizar esos recursos? Y ¿cómo establecer la calidad final del producto? De tal manera, este curso te permitirá conocer la industria del desarrollo de software.

LIBROS RECOMENDADOS

1

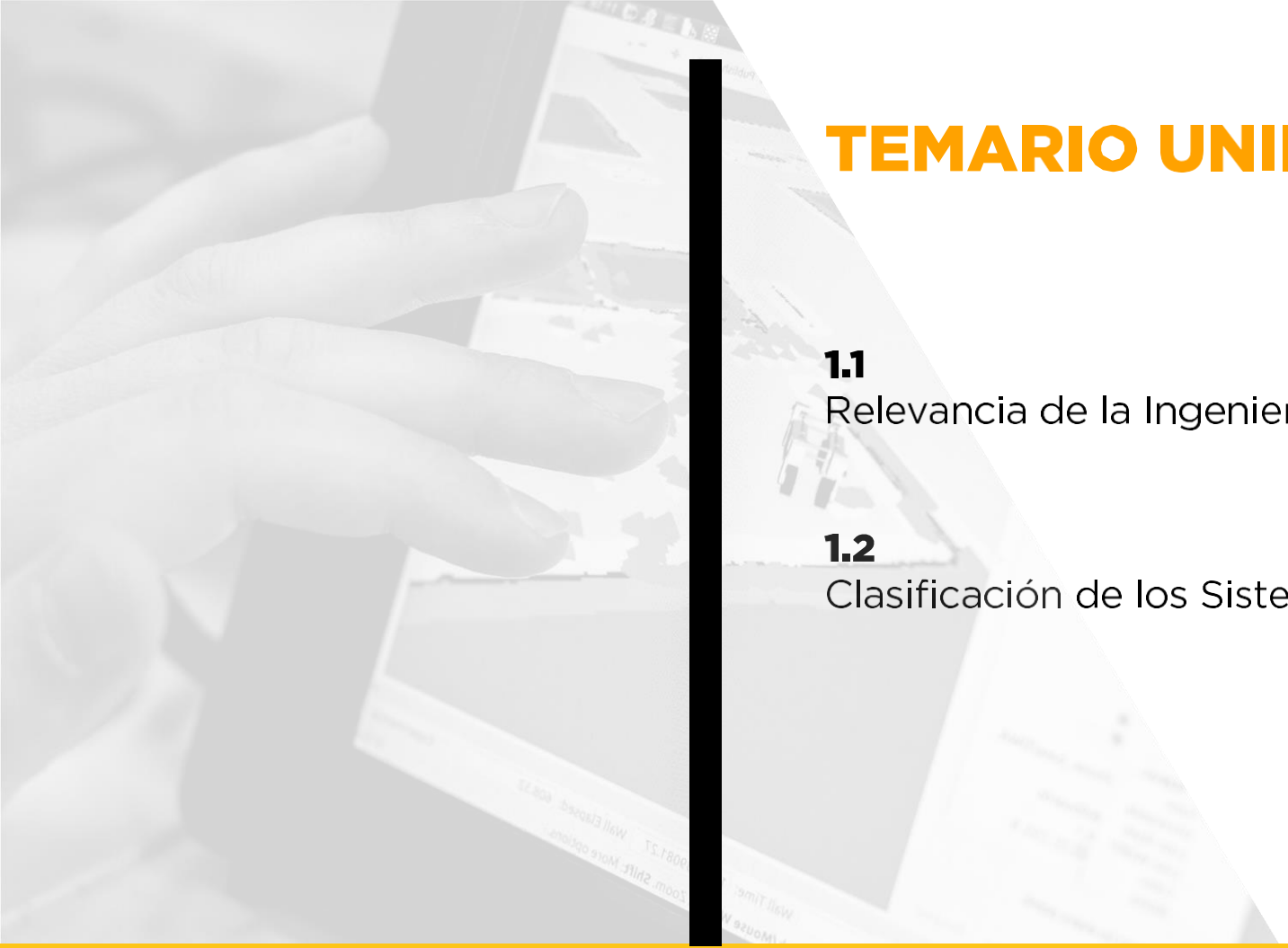
Stephens, R. (2015) Beginning Software Engineering. John Wiley & Sons, Inc.

2

Ahmed, A. & Prasad, B. (2016) Foundations of Software Engineering. CRC Press.

UNIDAD 1

LA INGENIERÍA DE SOFTWARE Y LOS SISTEMAS DE INFORMACIÓN



TEMARIO UNIDAD 1

1.1
Relevancia de la Ingeniería de Software

1.2
Clasificación de los Sistemas de Información

INTRODUCCIÓN

La asignatura *Ingeniería de Software I* tiene como objetivo introducir al alumno en el proceso de desarrollo de software. A diferencia de otros procesos productivos, el software es un producto intangible.

En esta primera unidad aprenderás sus conceptos básicos, su aplicación y los tipos de sistemas que existen.

COMPETENCIAS A DESARROLLAR



El alumno será capaz de:

- Explicar qué es la ingeniería de software y por qué es importante.

- Describir los sistemas de información, así como sus componentes y clasificaciones.

RELEVANCIA DE LA INGENIERÍA DE SOFTWARE

Para entender por qué la ingeniería de software es relevante, primero hay que entender **¿qué es el software?**

De acuerdo con Ian Sommerville (2005), el software no es solo un programa de computadora, sino todos los documentos asociados y la configuración de los datos que se necesitan para hacer que dicho programa opere de manera correcta.



Existen 2 tipos de productos de software:

- **Productos genéricos.** Son sistemas aislados que se venden al mercado abierto a cualquier cliente. Un ejemplo de este tipo de producto es el software para PC. Algunos softwares bajo esta categoría son: bases de datos, procesadores de texto, paquetes de diseño gráfico y herramientas de gestión de proyectos.

- **Productos personalizados (o hechos a la medida).** Son sistemas requeridos por un cliente en particular. Bajo esta premisa, un contratista de software los desarrolla específicamente para dicho cliente. Ejemplos de este tipo de software son los sistemas para el control de instrumentos electrónicos, sistemas para llevar a cabo algún proceso de negocios y sistemas de control del tráfico aéreo.

Una diferencia importante entre estos tipos de software es que, en los productos genéricos, la organización que desarrolla el software controla su especificación.

La especificación de los productos personalizados, por lo general, es desarrollada y controlada por la organización que compra el software. Los desarrolladores de software deben trabajar con esa especificación.



La clasificación anterior es importante porque define reglas distintas para la gestión del proceso de desarrollo. Sin embargo, la ingeniería de software se enfoca en ambos tipos.

En este sentido, la ingeniería de software es:

- "La aplicación de un enfoque sistemático, disciplinado y cuantificable para el desarrollo, operación y mantenimiento del software." [IEEE, 1993]

- "Un conjunto de etapas parcialmente ordenadas con la intención de lograr un objetivo, en este caso, la obtención de un producto de software de calidad." [Jacobson, 1998]

- "Es una disciplina de la ingeniería que comprende todos los aspectos de la producción de software, desde las etapas iniciales de la especificación del sistema hasta el mantenimiento después de que se utiliza." [Ian Somerville, 2005]

Así, la Ingeniería de Software ayuda a seguir procesos estandarizados para alcanzar los siguientes atributos de un buen software:

- **Mantenibilidad.** El software debe escribirse de tal forma que pueda evolucionar para cumplir las necesidades de cambio de los clientes. Este es un atributo crítico, debido a que el cambio en el software es una consecuencia inevitable, a su vez, de un cambio en el entorno de los negocios.

- **Confiabilidad.** El software confiable no debe causar daños físicos o económicos en el caso de una falla del sistema.

- **Eficiencia.** El software no debe hacer que se malgasten los recursos del sistema, como la memoria y los ciclos de procesamiento.

- **Usabilidad.** El software debe ser fácil de usar, sin esfuerzo adicional por parte del usuario. Por ello, se cuida su interfaz y su documentación.

La Ingeniería de Software es un área que, para mantenerse relevante, necesita afrontar todo tipo de retos constantemente. Algunos de estos son los siguientes:

• **Heterogeneidad.** Cada vez más, se requiere que los sistemas operen como sistemas distribuidos en redes que incluyen diferentes tipos de computadoras, con diferentes clases de sistemas de soporte y lenguajes de programación.





RELEVANCIA DE LA INGENIERÍA DE SOFTWARE

A menudo, es necesario integrar software nuevo con sistemas heredados más viejos. El reto de la heterogeneidad es desarrollar técnicas para construir software confiable que sea lo suficientemente flexible para adecuarse a esta heterogeneidad.

- **Entrega.** Consiste en reducir los tiempos de entrega para sistemas grandes y complejos, sin que se comprometa la calidad del sistema.



• **Confianza.** Tiene que ver con el desarrollo de distintas técnicas que demuestren a los usuarios que pueden confiar en el software porque es seguro, no tiene errores, está siempre disponible y protege nuestra privacidad, entre otros beneficios.

La investigación que propician los métodos de la ingeniería de software permite resolver este tipo de retos y desmentir mitos que se creían ciertos en el pasado.

VIDEO

Te invitamos a ver el siguiente video:



ALGUNOS DE ESTOS MITOS SON:

- **Mito.** Si fallamos en la planeación, podemos añadir más programadores y adelantar el tiempo perdido.

- **Realidad.** Cuando se añaden nuevas personas, la necesidad de aprender y comunicarse con el equipo puede y hace que se reduzca la cantidad de tiempo gastado en el desarrollo productivo. Pudiera añadirse más gente, pero de una manera planificada y bien coordinada.

- **Mito.** Una definición general de los objetivos es suficiente para empezar a escribir los programas (podemos detallar más adelante).

- **Realidad.** Una mala definición inicial es la causa del desperdicio del trabajo en software. Es esencial una descripción formal y detallada del ámbito de la información, funciones, rendimiento, etc. Además, todas estas características solo se pueden determinar después de una exhaustiva comunicación entre el cliente y el analista.

- **Mito.** Los requisitos del proyecto cambian continuamente, pero los cambios pueden realizarse fácilmente, ya que el software es flexible.

- **Realidad.** Es verdad que los requisitos del software cambian, pero el impacto del cambio varía según el momento en que se introduzca. Entre más temprano se realice, mejor.

- **Mito.** Lo único que se entrega al terminar es el programa funcionando.

- **Realidad.** Un programa que funciona es solo una parte de la configuración del software, la cual incluye elementos tales como el análisis, diseño, documentación del sistema, etcétera.

LECTURA

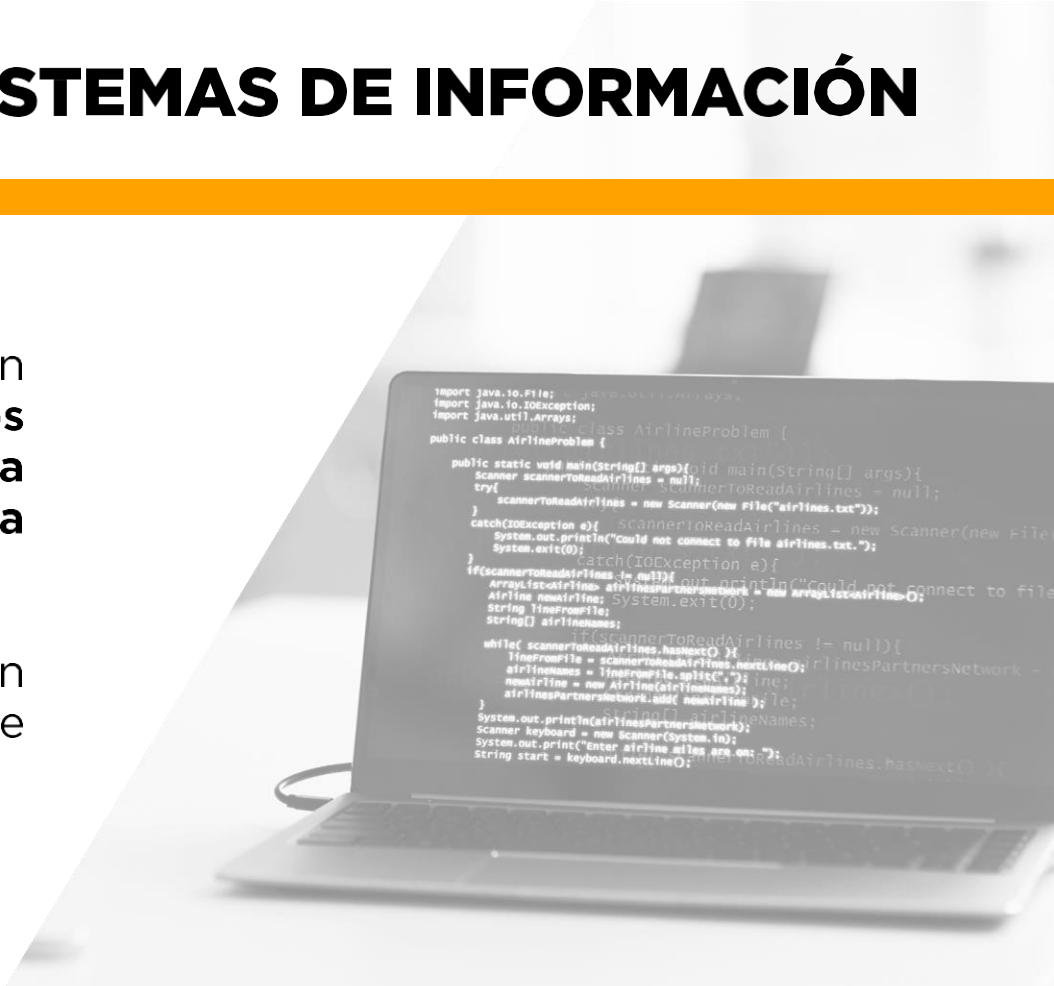
Introducción a la
Ingeniería de Software



CLASIFICACIÓN DE LOS SISTEMAS DE INFORMACIÓN

Un Sistema de Información (SI) es un **conjunto de elementos interrelacionados con el propósito de prestar atención a las demandas de información de una organización.**

Lo anterior, permite brindar soporte en la toma de decisiones y desarrollo de acciones.



Los componentes más importantes de un sistema de información son los que a continuación se exponen:

a) Financieros. Son los aspectos económicos que permiten la adquisición, contratación y mantenimiento de los demás recursos que integran un sistema de información.

b) Administrativos. Conforman la estructura orgánica de objetivos, lineamientos, funciones, procedimientos, departamentalización, dirección y control de las actividades; que sustenta la creación y uso de los sistemas.

c) Humanos. A su vez, compuestos por:

1) El técnico: quien posee los conocimientos especializados en el desarrollo de sistemas.

2) El usuario: representado por las personas interesadas en el manejo de información vía cómputo.

d) Materiales. Son aquellos elementos físicos que soportan el funcionamiento de un sistema de información.

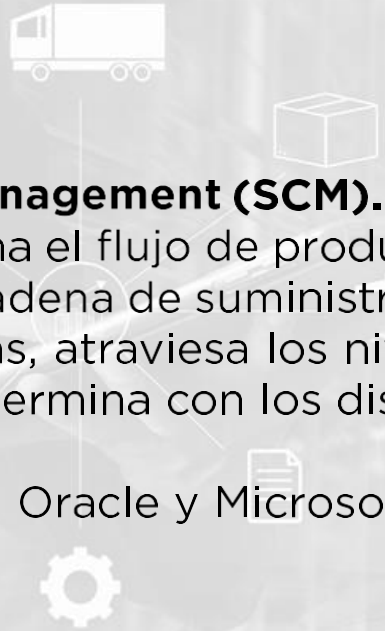
e) Tecnológicos. Son el conjunto de conocimientos, experiencias, metodologías y técnicas, que orientan la creación, operación y mantenimiento de un sistema.



De acuerdo al proceso de negocio que atienden los sistemas de información, pueden clasificarse de la siguiente manera:

- **Enterprise Resource Planning (ERP).** Es un acrónimo que significa planificación de recursos empresariales. Se trata de un software de gestión de procesos empresariales que gestiona e integra las actividades financieras, de cadena de suministro, operaciones, informes, fabricación y recursos humanos de una empresa.

SAP, Oracle y Microsoft son proveedores de soluciones ERP.



• **Supply chain management (SCM).** Un sistema de gestión de la cadena de suministro gestiona el flujo de productos, datos, dinero e información a lo largo de toda la cadena de suministro. Esta comienza con los proveedores de materias primas, atraviesa los niveles intermedios de las empresas de procesamiento y termina con los distribuidores y minoristas.

Nuevamente, SAP, Oracle y Microsoft lideran este tipo de soluciones.

- **Customer Relationship Management (CRM).** Es un enfoque para gestionar la interacción de una empresa con sus clientes actuales y potenciales. Utiliza el análisis de datos de la historia de los clientes con la empresa para mejorar sus relaciones comerciales. Se centra, específicamente, en la retención de los mismos y, en última instancia, en impulsar el crecimiento de las ventas.

Foro 1 – Unidad 1. Entorno de trabajo

Participa en el foro enviando imágenes que demuestren que ya tienes acceso a las siguientes herramientas en su versión de prueba:

- Alguna herramienta CASE (Computer Aided Software Engineering) para trabajar de manera colaborativa.

Presiona el botón para participar en el foro.



CONCLUSIÓN

La Ingeniería de Software es muy importante, ya que con ella se puede analizar, diseñar, programar e implementar un software de manera correcta y organizada. Todo esto siempre con la intención de cumplir con todas las especificaciones, tanto del cliente como del usuario final. También, suministra a los desarrolladores las bases para construir software de alta calidad en una forma eficiente. De la misma manera, define una disciplina que garantiza la producción y el mantenimiento del software, el cual debe ser desarrollado en el plazo fijado y dentro del costo estimado. Sus principales ventajas son:

- Mejorar la calidad de los productos de software.
- Aumentar la productividad y trabajo de los ingenieros del software.
- Facilitar el control del proceso de desarrollo de software.

¡Felicidades!

Acabas de concluir la **primera unidad** de tu curso **Ingeniería de Software I.** Te invitamos a finalizar este esfuerzo realizando el examen parcial correspondiente.

Para ello, debes regresar a la pantalla principal y dar clic en *Presentar examen*.

UNIDAD 2
MODELOS DE LA INGENIERÍA DE SOFTWARE

TEMARIO UNIDAD 2

2.1
Modelo de Madurez de Capacidades

2.2
Modelos de Desarrollo de Software

INTRODUCCIÓN

En esta segunda unidad veremos cómo una empresa dedicada al desarrollo de software puede mejorar para ser más profesional.

El modelo de madurez de capacidades y los modelos de desarrollo de software, que veremos en esta unidad, ayudan a las empresas a ubicarse y saber en dónde están y cómo mejorar.

COMPETENCIAS A DESARROLLAR



El alumno comprenderá:

- El modelo de madurez de capacidades de desarrollo del software.

El alumno conocerá:

- Los modelos de desarrollo de software más importantes y sus tendencias.

MODELO DE MADUREZ DE CAPACIDADES

El modelo de madurez de capacidades tiene como finalidad evaluar la madurez de los procesos de desarrollo de software en una organización. De la misma manera, se busca un plan de mejora de los procesos de desarrollo de software de acuerdo con 5 niveles de madurez.

También se le conoce como CMM *(Capability Maturity Model)* y fue desarrollado por la Universidad Carnegie-Mellon para el *Software Engineering Institute* (SEI).



VIDEO

Te invitamos a ver el siguiente video:



1

El modelo tiene cinco niveles que se describen a continuación:

- **Nivel 1 (inicial).** Las organizaciones en este nivel no disponen de un ambiente estable para el desarrollo y mantenimiento de software. Aunque se utilicen técnicas correctas de ingeniería, los esfuerzos se ven minados por falta de planificación. El éxito de los proyectos se basa en el esfuerzo personal. El resultado de los proyectos es impredecible.

2

• **Nivel 2 (repetible).** El proceso de desarrollo de software involucra planeación y seguimiento: esto está documentado. La empresa cuenta con políticas que le permiten administrar un proyecto de software y verifica cuáles son implementadas. El cliente puede saber el estado del proyecto al concluir cada una de sus etapas de desarrollo. Se refiere a poder repetir éxitos anteriores, con base en la experiencia y proyectos previos en donde se ha obtenido éxito.

3

• **Nivel 3 (definido).** Se cuenta con un proceso de software estándar. La organización capacita al personal en este proceso. Se implementan técnicas de revisión por pares (*peer reviews*). La probabilidad de acertar en los estimados es muy precisa.

4

- **Nivel 4 (gestionado).** Se caracteriza porque las organizaciones disponen de un conjunto de métricas significativas de calidad y productividad. Estas se usan sistemáticamente para tomar decisiones y gestionar riesgos.

5

• **Nivel 5 (optimizado).** La organización se dedica a mejorar continuamente su proceso. Se gestiona el proceso de innovación. Se ubican los orígenes de las fallas y se corrigen antes de que sea demasiado costoso para el proyecto.

VIDEO

Te invitamos a ver el siguiente video:



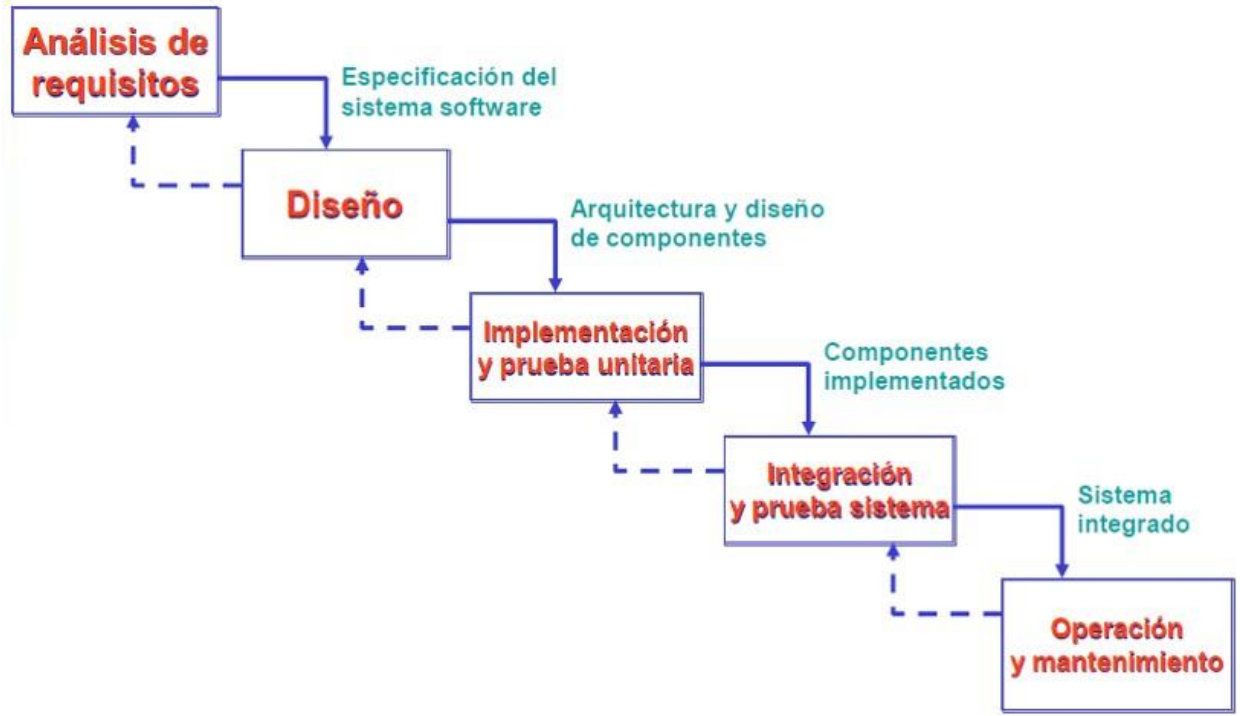
MODELOS DE DESARROLLO DE SOFTWARE

Un **marco de trabajo** o ***framework*** es una estructura genérica que puede ser extendida para crear un subsistema o aplicación más específica. Hay distintos tipos de marcos de trabajo, por ejemplo:

- El modelo en cascada,
- El modelo de prototipos evolutivos,
- El modelo en espiral y
- El proceso unificado (RUP).



El **modelo en cascada** surgió en los años 70 y consta de cinco fases secuenciales:





MODELO EN CASCADA

- **Análisis de requisitos.** Los servicios, restricciones y metas del sistema se definen a detalle a partir de las consultas con los usuarios.

- **Diseño.** El proceso de diseño del sistema divide los requerimientos en responsabilidades del hardware o software. Establece una arquitectura completa del sistema.



MODELO EN CASCADA

- **Implementación y prueba unitaria.** El diseño del software se lleva a cabo como un conjunto o unidades de programas. La prueba de unidades implica verificar que cada una cumpla su especificación.

- **Integración y prueba de sistema.** Las unidades individuales de programas se integran y se prueban como un sistema completo, para asegurar que se cumplan los requerimientos del software.



MODELO EN CASCADA

- **Operación y mantenimiento.** Por lo general (aunque no necesariamente), esta es la fase más larga del ciclo de vida. El sistema se instala y se pone en funcionamiento práctico. El mantenimiento implica corregir errores no descubiertos en las etapas anteriores del ciclo de vida, descubrir omisiones en los requerimientos originales, hacer mejoras al software y descubrir nuevos requerimientos.

MODELO EN CASCADA

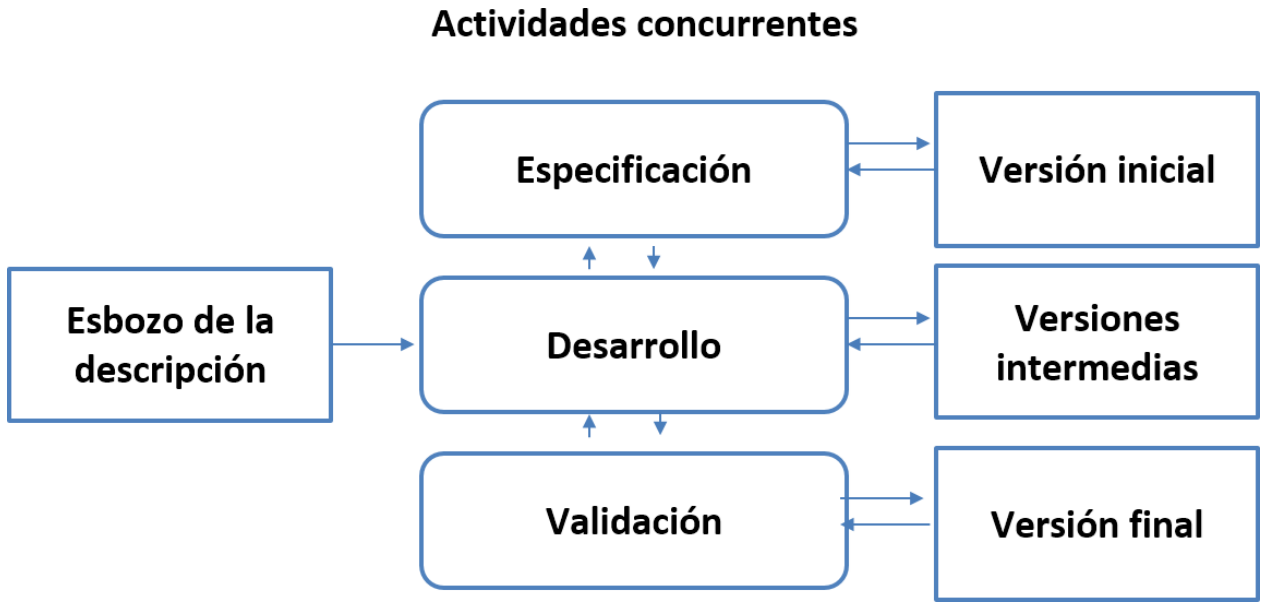
Ventaja

La principal ventaja del modelo en cascada es que la documentación se produce en cada fase y que este modelo cuadra con otros modelos del proceso de ingeniería.

Desventaja

Su principal problema es su inflexibilidad al dividir el proyecto en distintas etapas. Se deben hacer compromisos en las etapas iniciales, lo que hace difícil responder a los cambios en los requerimientos del cliente.

El **modelo de prototipos evolutivos** nace como un método para acelerar la definición de los requerimientos del software por construir:



MODELO DE PROTOTIPOS EVOLUTIVOS

Se basa en la idea de desarrollar un prototipo inicial, exponerlo a los comentarios del usuario y refinarlo a través de las distintas versiones, hasta que se desarrolla el sistema adecuado.

Las actividades de especificación, desarrollo y validación se entrelazan en vez de separarse, con una rápida retroalimentación entre estas.



MODELOS DE DESARROLLO DE SOFTWARE

MODELO DE PROTOTIPOS EVOLUTIVOS

Existen dos tipos de desarrollo evolutivo:

- **Desarrollo exploratorio.** El desarrollo empieza con las partes del sistema que se comprenden mejor. El sistema evoluciona agregando nuevos atributos propuestos por el cliente.

- **Prototipos desechables.** El objetivo del proceso es comprender los requerimientos del cliente. Se centra en experimentar con los requerimientos que no se comprenden del todo.

MODELO DE PROTOTIPOS EVOLUTIVOS

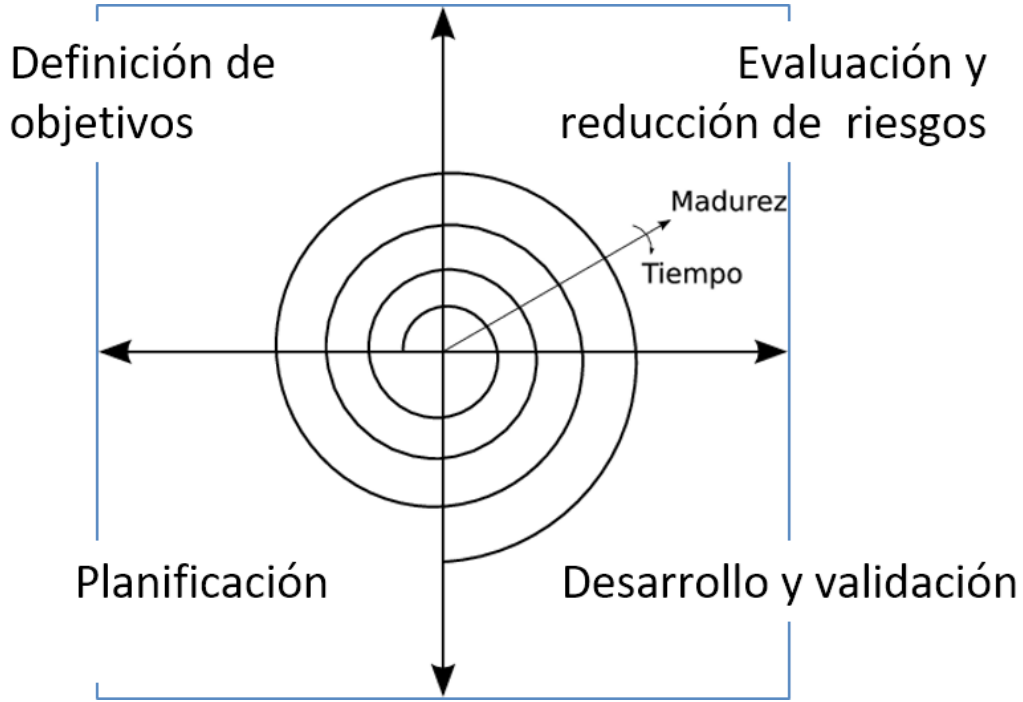
Ventaja

La ventaja del modelo de prototipos evolutivos es que la especificación se puede desarrollar en forma creciente. Tan pronto como los usuarios desarrollen un mejor entendimiento de su problema, este se puede reflejar en el sistema de software.

Desventaja

Al no establecerse claramente los alcances del producto, en cada revisión del mismo se podrían desbordar los requerimientos, lo cual provocaría un proyecto sin fin.

El **modelo en espiral** representa una fase del proceso del software en cada ciclo de la espiral. El primer ciclo es la viabilidad:





MODELO EN ESPIRAL

Cada ciclo de la espiral se divide en cuatro sectores:

1. Definición de objetivos. Para esta fase del proyecto se definen los objetivos específicos. Se identifican las restricciones del proceso y el producto, y se traza un plan detallado de gestión. Se identifican los riesgos del proyecto. Dependiendo de estos riesgos, se planean estrategias alternativas.



MODELO EN ESPIRAL

2. Evaluación y reducción de riesgos. Se lleva a cabo un análisis detallado para cada uno de los riesgos del proyecto identificados. Se definen los pasos para reducirlos. Por ejemplo, si existe el riesgo de tener requerimientos inapropiados, se puede desarrollar un prototipo del sistema.



MODELO EN ESPIRAL

3. Desarrollo y validación. Después de la evaluación de riesgos, se elige un modelo para el desarrollo del sistema.

Por ejemplo, si los riesgos en la interfaz de usuario son dominantes, el uso de un modelo prototipo evolutivo podría ser apropiado. Si el mayor riesgo es la integración de los subsistemas, el modelo cascada puede ser el más apropiado, etc.



MODELO EN ESPIRAL

4. Planificación. El proyecto se revisa y se toma la decisión acerca de si se debe continuar con un ciclo posterior de la espiral. Si se decide continuar, se desarrollan los planes para la siguiente fase.

MODELO EN ESPIRAL

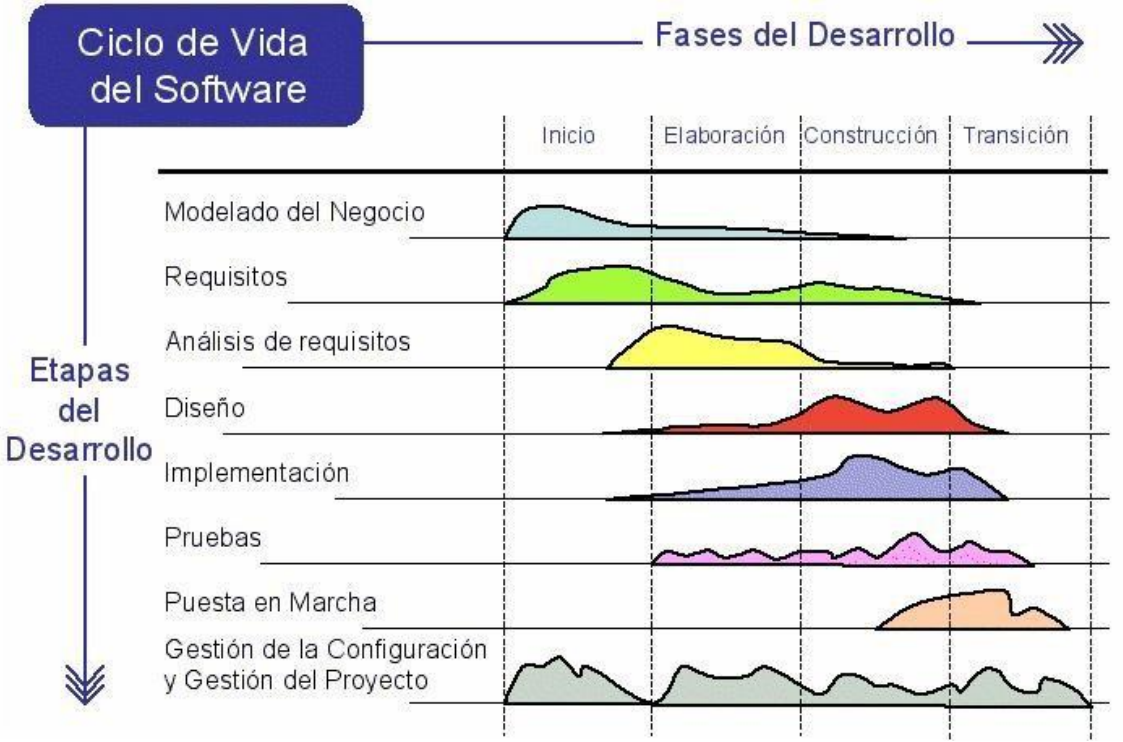
Ventaja

La ventaja del modelo de espiral es que demanda una consideración directa de los riesgos técnicos en todas las etapas del proyecto. Si se aplica adecuadamente, se deben reducir los riesgos antes de que se conviertan en problemas.

Desventaja

Si su administración pide un desarrollo apegado al presupuesto, el modelo en espiral se convierte en un problema. El costo se revisa y modifica cada vez que se termina un circuito.

El **modelo de proceso unificado (RUP)** es el estándar más utilizado en el paradigma orientado a objetos:





MODELO DE PROCESO UNIFICADO (RUP)

El RUP propone 4 fases de desarrollo de software:

1. Inicio. Establece un caso de negocio para el sistema. Se deben identificar todas las entidades externas (personas y sistemas) que interactuarán con el sistema y definir estas interacciones. Esta información se utiliza entonces para evaluar la aportación que el sistema hace al negocio.



MODELO DE PROCESO UNIFICADO (RUP)

2. Elaboración. Al terminar esta fase, se debe tener un modelo de los requerimientos del sistema (se especifican los casos de uso UML), una descripción arquitectónica y un plan de desarrollo del software.

3. Construcción. Durante esta fase se desarrollan e integran las partes del sistema. Al terminar, debe tener un sistema de software operativo y la documentación correspondiente lista para entregarla a los usuarios.



MODELO DE PROCESO UNIFICADO (RUP)

4. Transición. La fase final del RUP se ocupa de mover el sistema desde la comunidad de desarrollo a la comunidad del usuario, y hacerlo trabajar en un entorno real. Esto se deja de lado en la mayor parte de los modelos de procesos del software pero es, en realidad, una actividad de alto costo y a veces problemática. Al terminar esta fase, se debe tener un sistema software documentado que funcione correctamente en su entorno operativo.



MODELO DE PROCESO UNIFICADO (RUP)

Las 4 fases mencionadas anteriormente son soportadas por **7 etapas de desarrollo:**

- **Modelado del negocio.** Los procesos del negocio se modelan utilizando casos de uso de negocio.

- **Requisitos.** Se definen los actores que interactúan con el sistema y se desarrollan casos de uso para modelar los requerimientos del sistema.



MODELO DE PROCESO UNIFICADO (RUP)

- **Análisis de requisitos.** Se crea y documenta un modelo del diseño utilizando modelos arquitectónicos, modelos de componentes, modelos de objetos y modelos de secuencias.

- **Implementación.** Se implementan y estructuran en subsistemas los componentes del sistema. La generación automática de código de los modelos del diseño ayuda a acelerar este proceso.



MODELO DE PROCESO UNIFICADO (RUP)

- **Pruebas.** Estas son un proceso iterativo que se lleva a cabo conjuntamente con la implementación. A la finalización de esta, tienen lugar las pruebas del sistema.

- **Puesta en marcha.** Se libera una versión del producto, se distribuye a los usuarios y se instala en su lugar de trabajo.

- **Gestión de la configuración y gestión del proyecto.**



MODELOS DE DESARROLLO DE SOFTWARE

MODELO DE PROCESO UNIFICADO (RUP)

RUP recomienda **6 buenas prácticas fundamentales:**

- **Desarrolla el software de forma iterativa.** Planifica incrementos del sistema basados en las prioridades del usuario. Entrega lo prioritario al inicio del proceso de desarrollo.

- **Verifica la calidad del software.** Asegura que el software cumple los estándares de calidad organizacionales.



MODELO DE PROCESO UNIFICADO (RUP)

- **Controla los cambios del software.**

- **Gestiona los requerimientos.** Documenta explícitamente los requerimientos del cliente y mantente al tanto de los cambios de estos requerimientos. Analiza el impacto de los cambios en el sistema antes de aceptarlos.



MODELOS DE DESARROLLO DE SOFTWARE

MODELO DE PROCESO UNIFICADO (RUP)

- **Utiliza arquitecturas basadas en componentes.**

- **Modela el software visualmente.** Utiliza modelos gráficos UML para presentar vistas estáticas y dinámicas del software.

MODELO DE PROCESO UNIFICADO (RUP)

Ventaja

La ventaja del modelo de proceso unificado es que identifica problemas oportunamente y el tiempo de desarrollo requerido es menor debido a la reutilización de componentes.

Desventaja

RUP es un modelo bastante complicado. Dada la variedad de componentes involucrados (mejores prácticas, fases, bloques de construcción, criterios de hitos, iteraciones y flujos de trabajo), la implementación puede ser un desafío.

LECTURA

Modelo de capacidades
de madurez



Aquellos métodos que se encuentran bajo la **filosofía ágil** se consideran marcos de trabajo distintos a los anteriores. Esto porque la comunidad del software identificó algunos **problemas en la ingeniería del software convencional:**

- Los proyectos son planificados en fases lineales y secuenciales, con un orden inalterable de precedencias y sin posibilidad de volver atrás de una fase a otra para realizar cambios sin agilidad.

- Se genera documentación exhaustiva. En ocasiones, se pasa más tiempo pensando cómo se debe desarrollar el sistema que en realizar el desarrollo y las pruebas. Cuando cambian los requerimientos, se retrabaja en la especificación y el diseño.

• No siempre proporcionan una retroalimentación temprana y continua al cliente durante el desarrollo del software para garantizar su satisfacción.

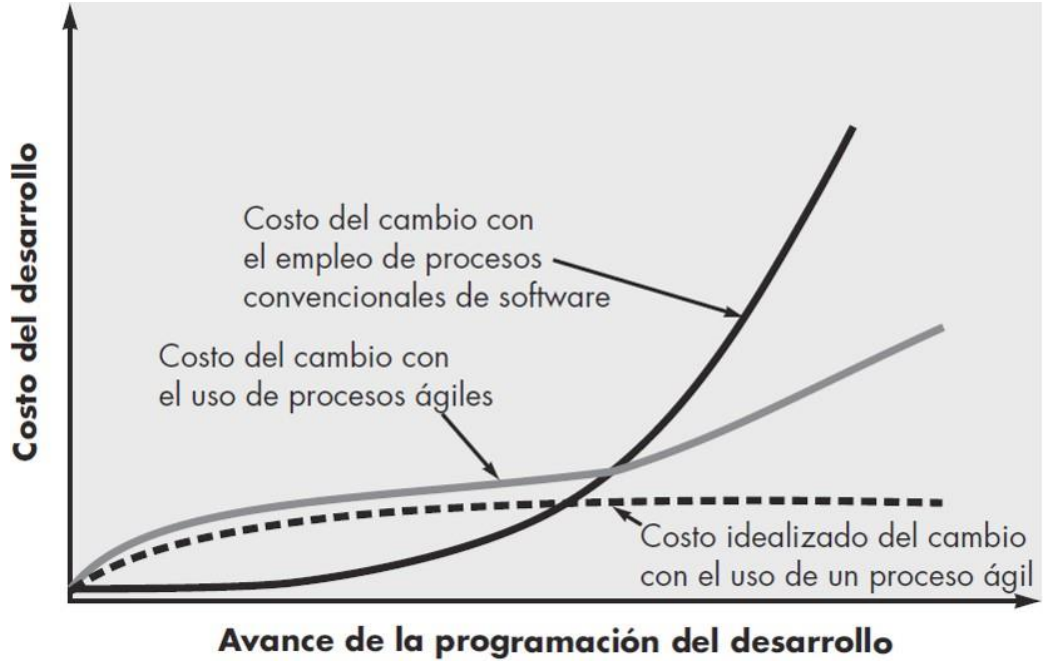
Los métodos ágiles se desarrollaron como un esfuerzo por superar las debilidades reales y percibidas de la ingeniería de software convencional. Para eso se desarrolló el siguiente **manifiesto ágil:**

Estamos descubriendo formas mejores de desarrollar software,
por medio de hacerlo y de dar ayuda a otros para que lo hagan.
Ese trabajo nos ha hecho valorar:

***Los individuos y sus interacciones,** sobre los procesos y las herramientas.*
***El software que funciona,** más que la documentación exhaustiva.*
***La colaboración con el cliente,** y no tanto la negociación del contrato.*
***Responder al cambio,** mejor que apegarse a un plan.*

Es decir, si bien son valiosos los conceptos que aparecen en
segundo lugar, valoramos más los que aparecen en primer sitio.

Una de las características más atractivas del enfoque ágil es su capacidad de reducir los costos del cambio durante el proceso del software.



La Alianza Ágil define 12 principios de agilidad:

1

La prioridad más alta es satisfacer al cliente a través de la entrega pronta y continua de software valioso.

2

Son bienvenidos los requerimientos cambiantes, aun en una etapa avanzada del desarrollo. Los procesos ágiles dominan el cambio para provecho de la ventaja competitiva del cliente.

3

Entregar con frecuencia software que funcione, de dos semanas a un par de meses, de preferencia lo más pronto que se pueda.

4

Las personas de negocios y los desarrolladores deben trabajar juntos, a diario y durante todo el proyecto.

5

Hay que desarrollar los proyectos con individuos motivados. Debe darse a estos el ambiente y el apoyo que necesiten, y confiar en que harán el trabajo.

6

El método más eficiente y eficaz para transmitir información a los integrantes de un equipo de desarrollo, y entre ellos, es la conversación cara a cara.

7

La medida principal de avance es el software que funciona.

8

Los procesos ágiles promueven el desarrollo sostenible. Los patrocinadores, desarrolladores y usuarios deben poder mantener un ritmo constante en forma indefinida.

9

La atención continúa a la excelencia técnica, y el buen diseño mejora la agilidad.

10

Es esencial la simplicidad: el arte de maximizar la cantidad de trabajo no realizado.

11

Las mejores arquitecturas, requerimientos y diseños surgen de los equipos con organización propia.

12

El equipo reflexiona a intervalos regulares sobre cómo ser más eficaz, para después afinar y ajustar su comportamiento en consecuencia.

MÉTODOS ÁGILES

Ventaja

Los métodos ágiles son los más idóneos para el desarrollo de sistemas de negocio pequeños y de tamaño medio, para el desarrollo de productos para computadoras personales.

Desventaja

No son adecuados para el desarrollo de sistemas a gran escala donde puedan haber complejas interacciones con otros sistemas hardware o software. Ni tampoco sistemas críticos que necesitan un análisis detallado de requerimientos.

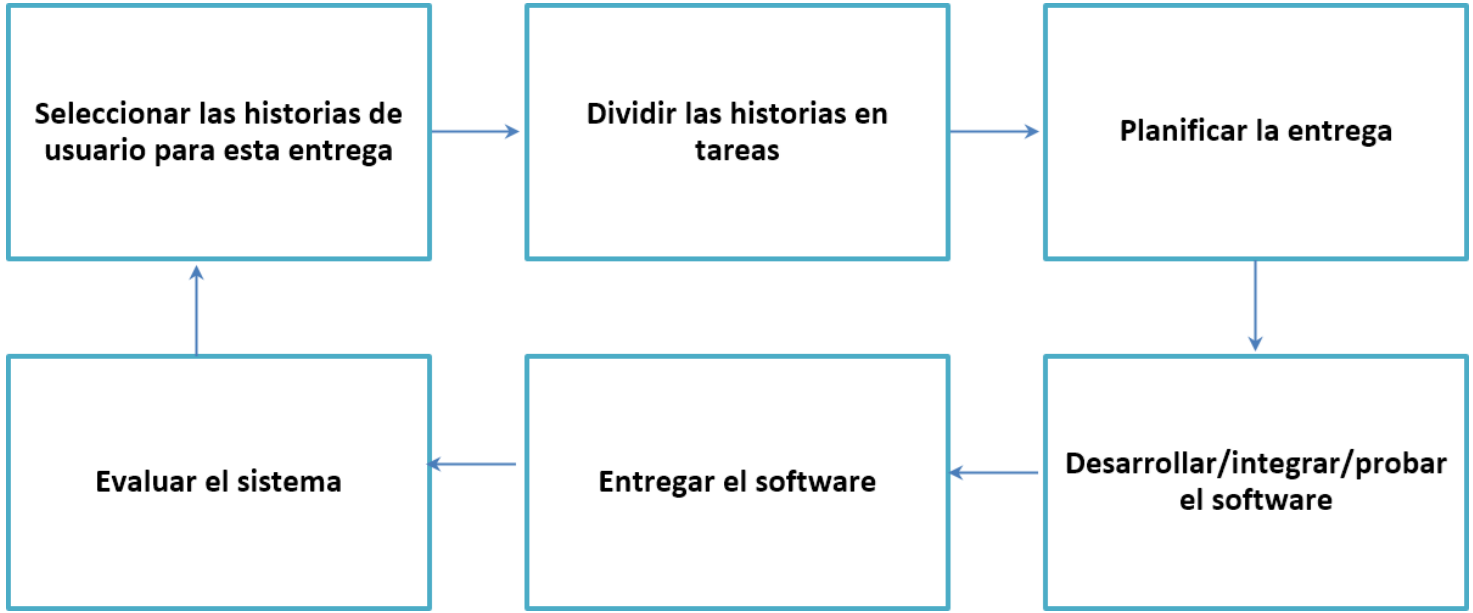


MODELOS DE DESARROLLO DE SOFTWARE

Algunos ejemplos de métodos que siguen la filosofía ágil son los siguientes:

- Programación extrema
- SCRUM

En la **programación extrema,** los programadores trabajan en parejas. El ciclo de la programación extrema es el siguiente:



Las características del proceso de la programación extrema son las siguientes:

• **Planificación incremental.** Los requerimientos se registran en tarjetas de historias a incluir en una entrega. Estas se determinan según el tiempo disponible y su prioridad relativa. Los desarrolladores se reparten estas historias en "tareas de desarrollo".

• **Entregas pequeñas.** El mínimo conjunto útil de funcionalidad que proporcione valor de negocio se desarrolla primero. Las entregas del sistema son frecuentes e incrementalmente añaden funcionalidad a la primera entrega.

• **Diseño sencillo.** Solo se lleva a cabo el diseño necesario para cumplir los requerimientos actuales.

• **Desarrollo previamente probado.** Se utiliza un sistema de pruebas de unidad automatizado para escribir pruebas para nuevas funcionalidades antes de que estas se implementen.

• **Refactorización.** Se espera que todos los desarrolladores refactoricen el código continuamente, tan pronto como se encuentren disponibles mejoras en este. Esto conserva el código sencillo y mantenible.

- **Programación en parejas.** Los desarrolladores trabajan en parejas, verificando cada uno el trabajo del otro y proporcionando la ayuda necesaria.

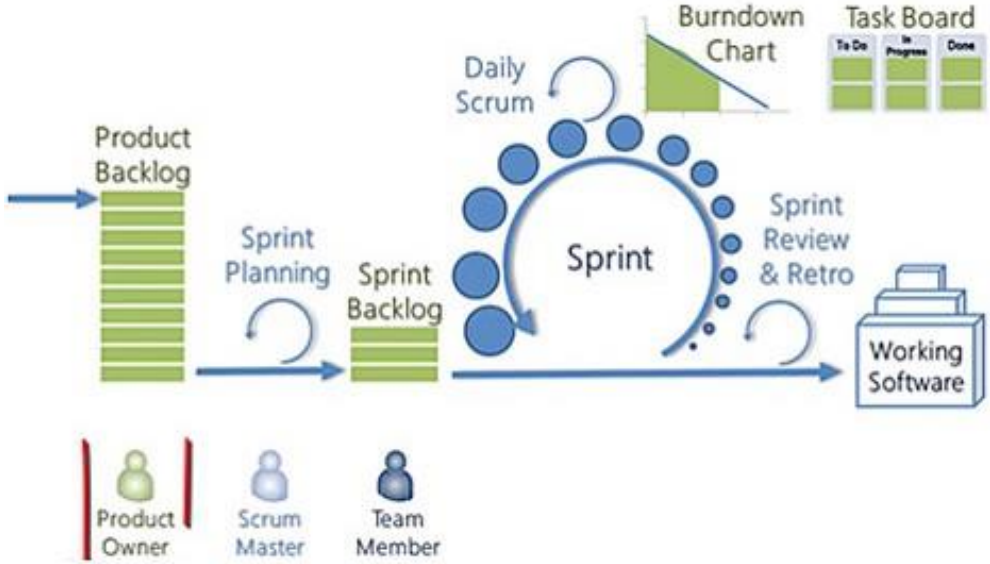
- **Integración continua.** En cuanto acaba el trabajo de una tarea, se integra en el sistema entero. Después de la integración, se deben pasar al sistema todas las pruebas de unidad.

- **Propiedad colectiva.** Las parejas de desarrolladores trabajan en todas las áreas del sistema, de modo que no desarrollen islas de conocimientos y todos los desarrolladores posean todo el código. Cualquiera puede cambiar cualquier cosa.

• **Ritmo sostenible.** No se consideran aceptables grandes cantidades de horas extras, ya que a menudo el efecto que tienen es que se reduce la calidad del código y la productividad a medio plazo.

•**Cliente presente.** Debe estar disponible en el equipo de programación extrema un representante de los usuarios finales del sistema (el cliente) a tiempo completo. El cliente es miembro del equipo de desarrollo y es el responsable de formular los requerimientos del sistema para su implementación. También define la prioridad de las historias de usuario.

El proceso **SCRUM** se comenzará siempre con aquello que le reporta un mayor beneficio al cliente. El modelo es el siguiente:



El proceso del modelo SCRUM es el siguiente:

- Inicia con los requisitos del producto o ***Product backlog.*** Este resume las tareas, funcionalidades y objetivos que persigue el proyecto.

- Posteriormente, en una reunión llamada ***sprint planning,*** se establecen entregas parciales. Estas son ordenadas y priorizadas.

- Cada entrega parcial de algo funcional se conoce internamente con el nombre de ***sprint.*** Estas entregas cuentan con una fecha de inicio y otra de fin. La duración aproximada suele ser de entre tres y cuatro semanas.

• Todos los días se tiene una reunión de 15 minutos llamada ***Daily SCRUM,*** donde cada miembro del equipo reporta qué hizo ayer, qué hará hoy y si hay algún problema que esté bloqueando el avance.

• Durante el *sprint*, las tareas evolucionarán de su estado inicial "por hacer" ***(to do)*** a "en progreso" ***(doing)*** y, finalmente, a "hecha" ***(done).*** Esperando que al final del *sprint* todas las tareas se hayan terminado.

• Cada que termina un *sprint* se tienen dos reuniones: *sprint review*, donde se muestra lo que se ha desarrollado; y *sprint retrospective*, donde se analiza qué se puede mejorar.

• Al final del *sprint*, se entrega un incremento de software al cliente.

Además del equipo de desarrollo, SCRUM incluye los siguientes roles:

- ***Product Owner.*** Es el único responsable de añadir tareas al ***Product backlog,*** para ordenarlas y organizarlas.

- ***Scrum Master.*** Su principal función es eliminar barreras y trabas que impiden al equipo de trabajo alcanzar los objetivos finales.

El ***Scrum Master*** y el ***Product Owner*** gestionan el cambio.

MODELO SCRUM

Ventaja

Una de las ventajas de SCRUM es que cada etapa del proceso arroja una serie de resultados. No es necesario, por tanto, que el cliente espere hasta el final para ver el resultado.

Desventaja

SCRUM funciona bien solo en equipos pequeños. Es difícil aplicarlo en grandes proyectos. Además, es necesario que el equipo de trabajo sea autoorganizado. Supone que el equipo está muy formado y motivado.

Actividad 1 – Unidad 2. Herramientas CASE

Te invitamos a realizar la siguiente actividad:

Presiona el botón para descargar la actividad



Presiona el botón para entregar la actividad



CONCLUSIÓN

Se estudiaron cuatro modelos convencionales de ingeniería del software: **1)** el modelo en cascada, que indica un desarrollo secuencial; **2)** el modelo de prototipos evolutivos, el cual ayuda a acelerar la definición de requerimientos a un bajo costo; **3)** el modelo de espiral, que tiene como enfoque el desarrollo incremental del software y que permite controlar el riesgo de las desviaciones del proyecto; y **4)** el RUP, que se caracteriza por estar centrado en la arquitectura y enfocarse al paradigma orientado a objetos.

Además, también se abordaron otras dos metodologías ágiles: **1)** El modelo de programación extrema, el cual se desarrolla principalmente en parejas; y **2)** SCRUM, en donde el trabajo se divide en *sprints* cortos que, al finalizarse, tienen un producto tangible.

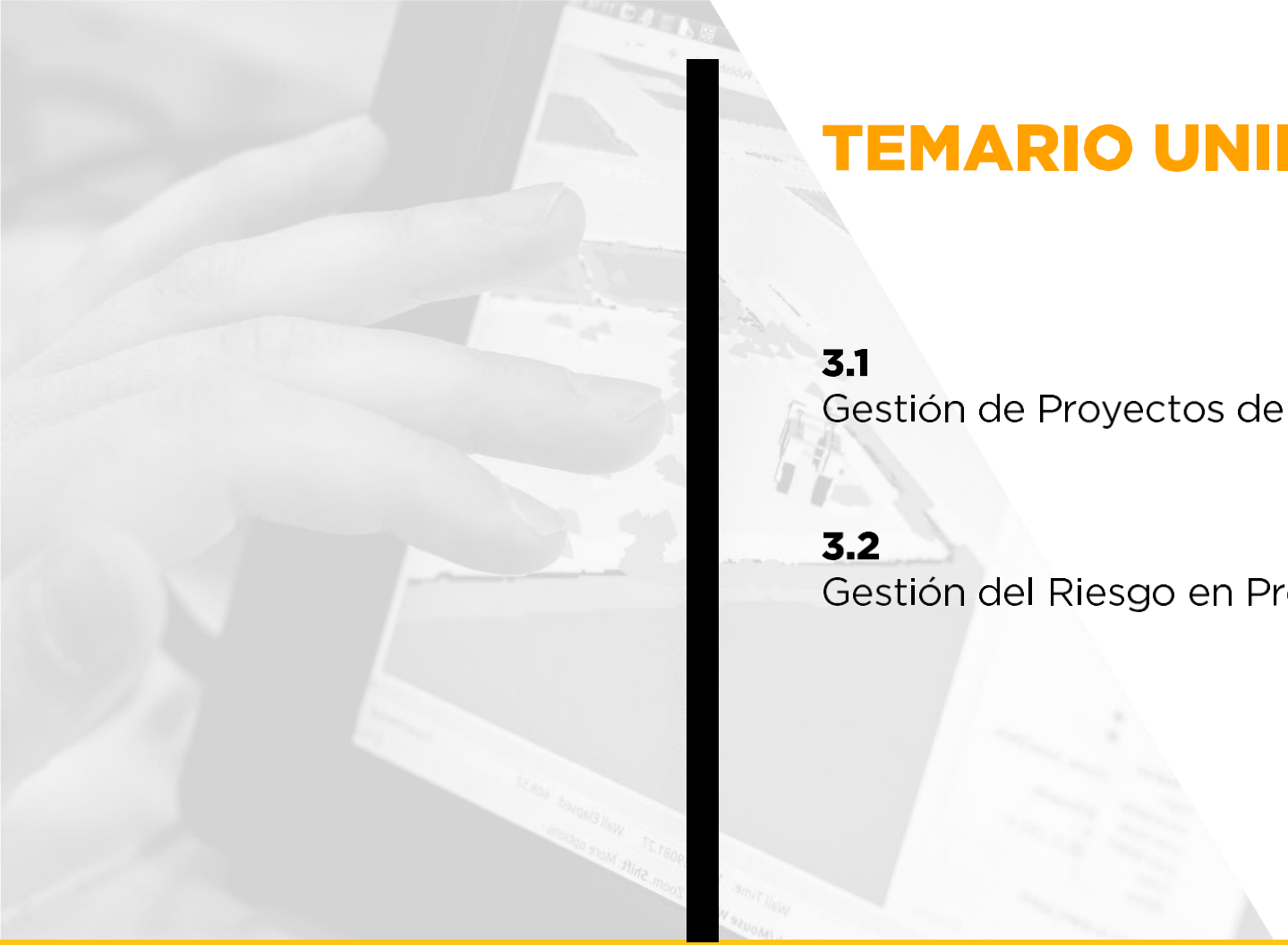
¡Felicidades!

Acabas de concluir la **segunda unidad** de tu curso **Ingeniería de Software I.** Te invitamos a finalizar este esfuerzo realizando el examen parcial correspondiente.

Para ello, debes regresar a la pantalla principal y dar clic en *Presentar examen*.



UNIDAD 3
PLANIFICACIÓN DEL PROYECTO DE SOFTWARE



TEMARIO UNIDAD 3

3.1
Gestión de Proyectos de Software

3.2
Gestión del Riesgo en Proyectos de Software

INTRODUCCIÓN

En esta tercera unidad se explica el por qué una buena administración no puede garantizar el éxito del proyecto. Sin embargo, una mala administración usualmente lleva a su fracaso, ya que, por ejemplo, el software es entregado tarde, los costos son mayores que los estimados y los requerimientos no se cumplen.

Por ello, la Ingeniería de Software requiere de administradores de proyectos de software que planifiquen y calendaricen el desarrollo de los mismos.

COMPETENCIAS A DESARROLLAR

El alumno comprenderá:

- Las principales tareas de los administradores de proyectos.

El alumno conocerá:

- Los principales métodos para estimar tiempos y costos en el desarrollo de software.



GESTIÓN DE PROYECTOS DE SOFTWARE

Una vez que se ha elegido el método de desarrollo de software se deben clarificar los siguientes temas:

- Alcance del proyecto
- Plan del proyecto
- Hitos del proyecto
- Calendarización del proyecto
- Costos del proyecto



El **alcance del software** deberá estar bien delimitado. Aquí se deben indicar datos concretos, las limitaciones que pudieran surgir y las facilidades de las que se va a disponer. En particular, se deben describir los siguientes **cinco aspectos**:

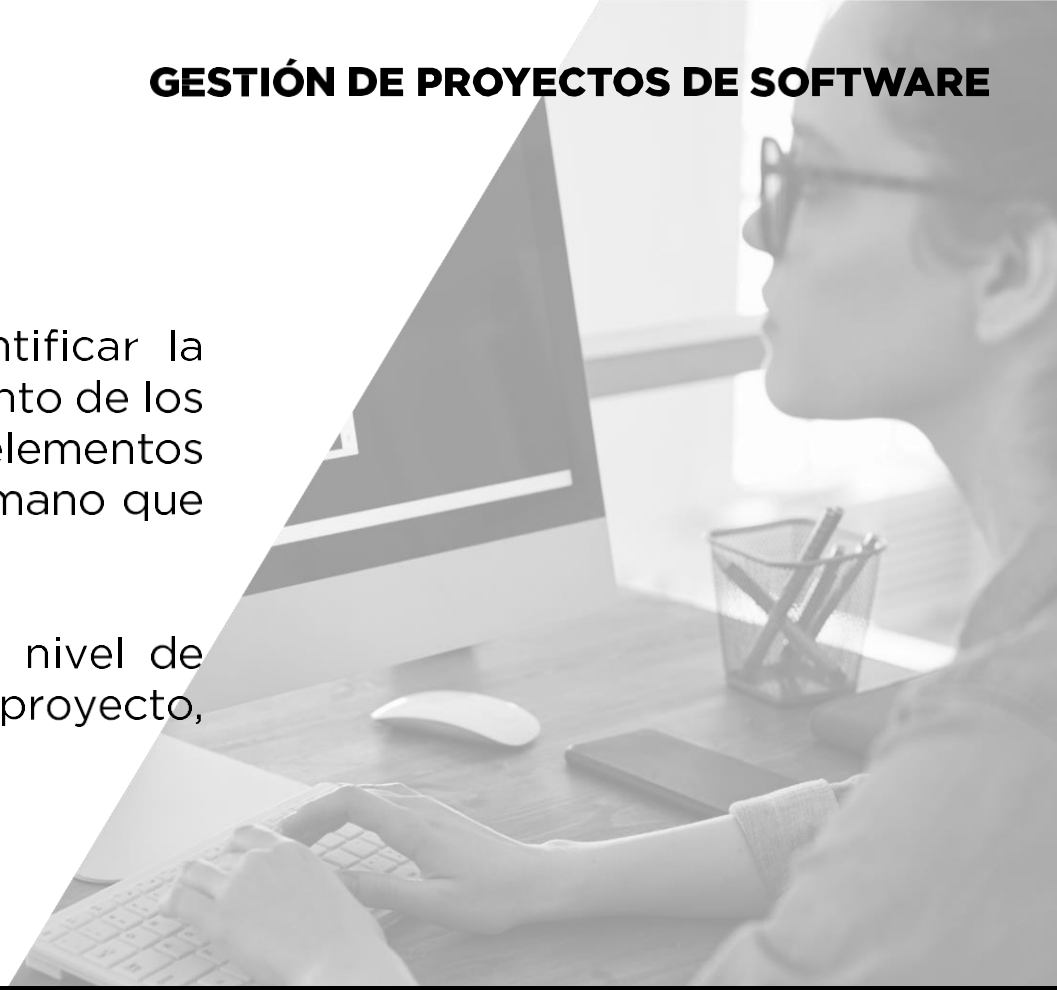
- **La función.** Se revisará y se concretarán las descritas en la ingeniería del sistema.

- **Rendimiento.** Necesidades en cuanto a tiempos de respuesta y procesamiento.

- **Las restricciones.** Son las limitaciones del hardware con que se encontrará el software.

- **Las interfaces.** Se deberá identificar la información que se comunicará, tanto de los dispositivos hardware como de los elementos software, así como el personal humano que hará uso de estos.

- **La fiabilidad.** Se establecerá el nivel de seguridad que deberá tener el proyecto, dependiendo de su naturaleza.



Después de definir el alcance del proyecto, se puede iniciar con la definición del plan. El **plan del proyecto** fija los recursos disponibles, divide el trabajo y crea un calendario de trabajo. Muchos planes incluyen las siguientes secciones:

- **Introducción.** Describe los objetivos del proyecto y expone las restricciones (por ejemplo, presupuesto, tiempo, etcétera) que afectan a la gestión del mismo.

- **Organización del proyecto.** Describe la forma en que el equipo de desarrollo está organizado, así como la gente involucrada y sus roles.

- **Análisis de riesgo.** Describe la probabilidad de que surjan riesgos y las estrategias para reducirlos.

• **Requerimientos de hardware y software.** Describe el hardware y el software de ayuda requeridos para llevar a cabo el desarrollo. Si es necesario comprar hardware, se deben incluir las estimaciones de los precios y las fechas de entrega.

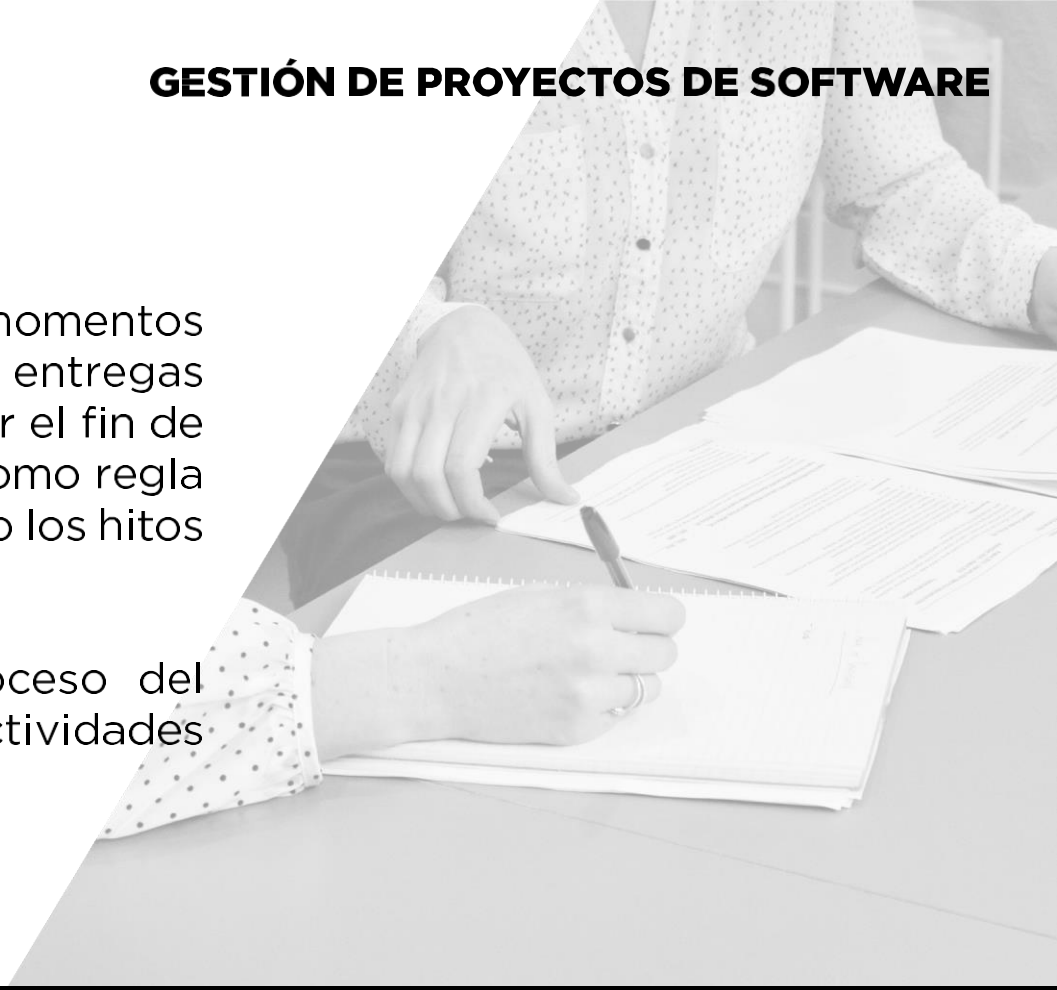
• **División del trabajo.** Describe la división del proyecto en actividades e identifica los hitos y productos a entregar asociados con cada actividad.

• **Programa del proyecto.** Describe las dependencias entre actividades, el tiempo estimado requerido para alcanzar cada hito y la asignación de personas.

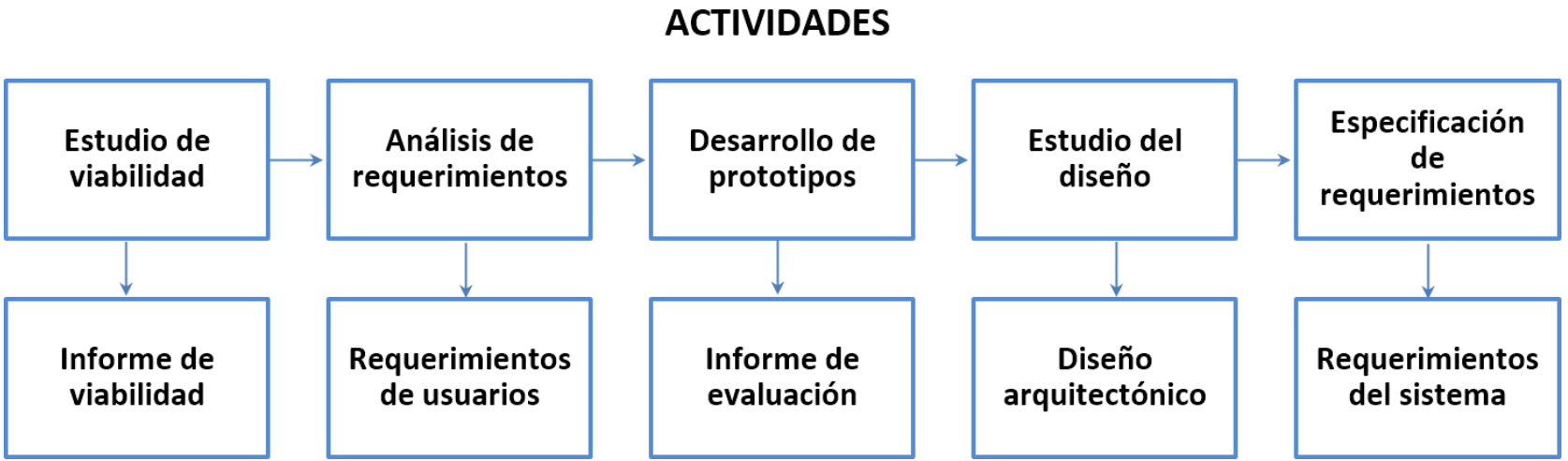
• **Mecanismos de supervisión e informe.** Describe la gestión de informes y fechas, así como los mecanismos de supervisión.

Los **hitos del proyecto** son los momentos clave en que se deben realizar entregas concretas. Estos deben representar el fin de una etapa lógica en el proyecto. Como regla general, las entregas son hitos, pero los hitos no son necesariamente entregas.

Para establecer los hitos, el proceso del software debe dividirse en actividades básicas con sus salidas asociadas.

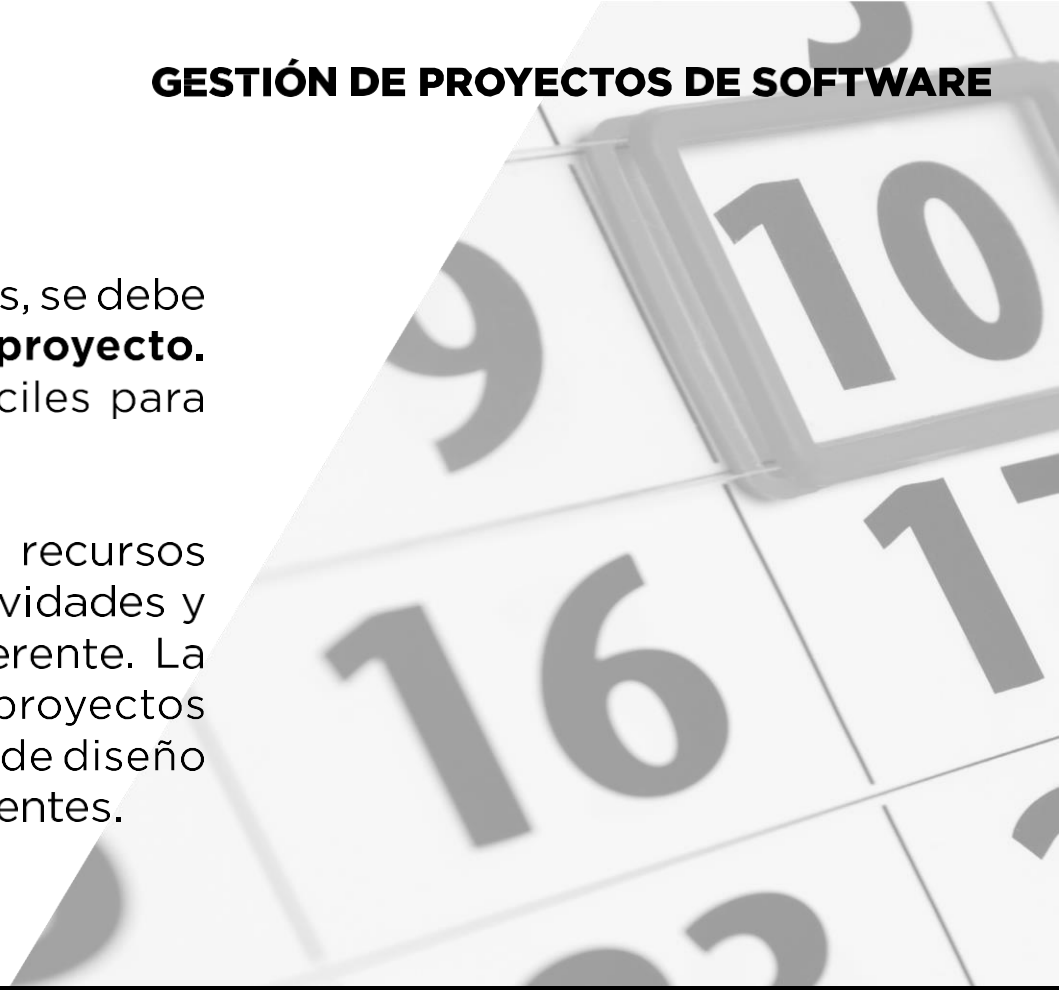


La siguiente figura muestra los hitos y actividades involucradas en la especificación de requerimientos:



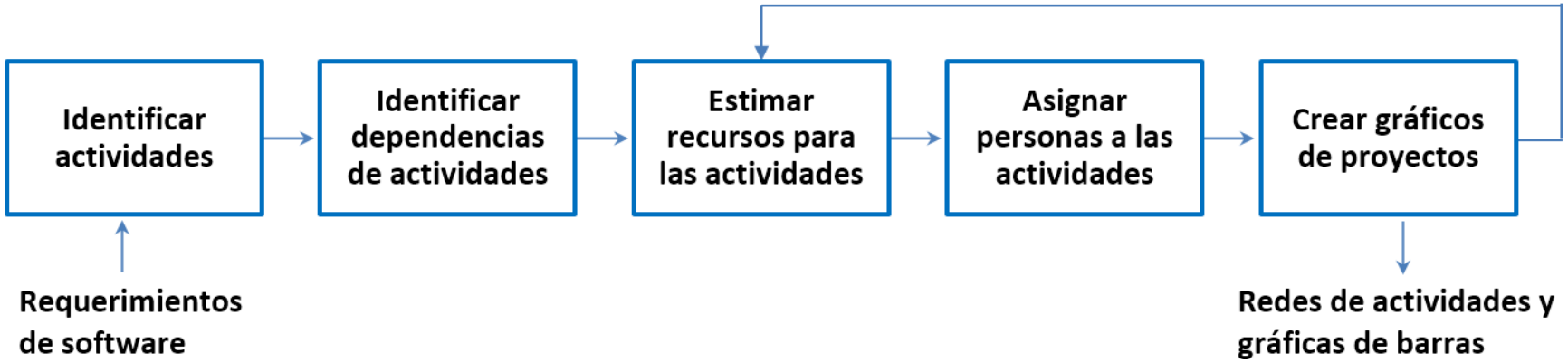
Una vez que se han definido los hitos, se debe establecer la **calendarización del proyecto.** Esta es una de las tareas más difíciles para los gestores de proyectos.

Ellos estiman el tiempo y los recursos requeridos para completar las actividades y organizarlas en una sucesión coherente. La estimación se complica porque, proyectos diferentes pueden utilizar métodos de diseño y lenguajes de programación diferentes.



La calendarización del proyecto implica separar todo el trabajo de un proyecto en actividades complementarias, y considerar el tiempo requerido para completar dichas actividades.

El proceso de calendarización del proyecto



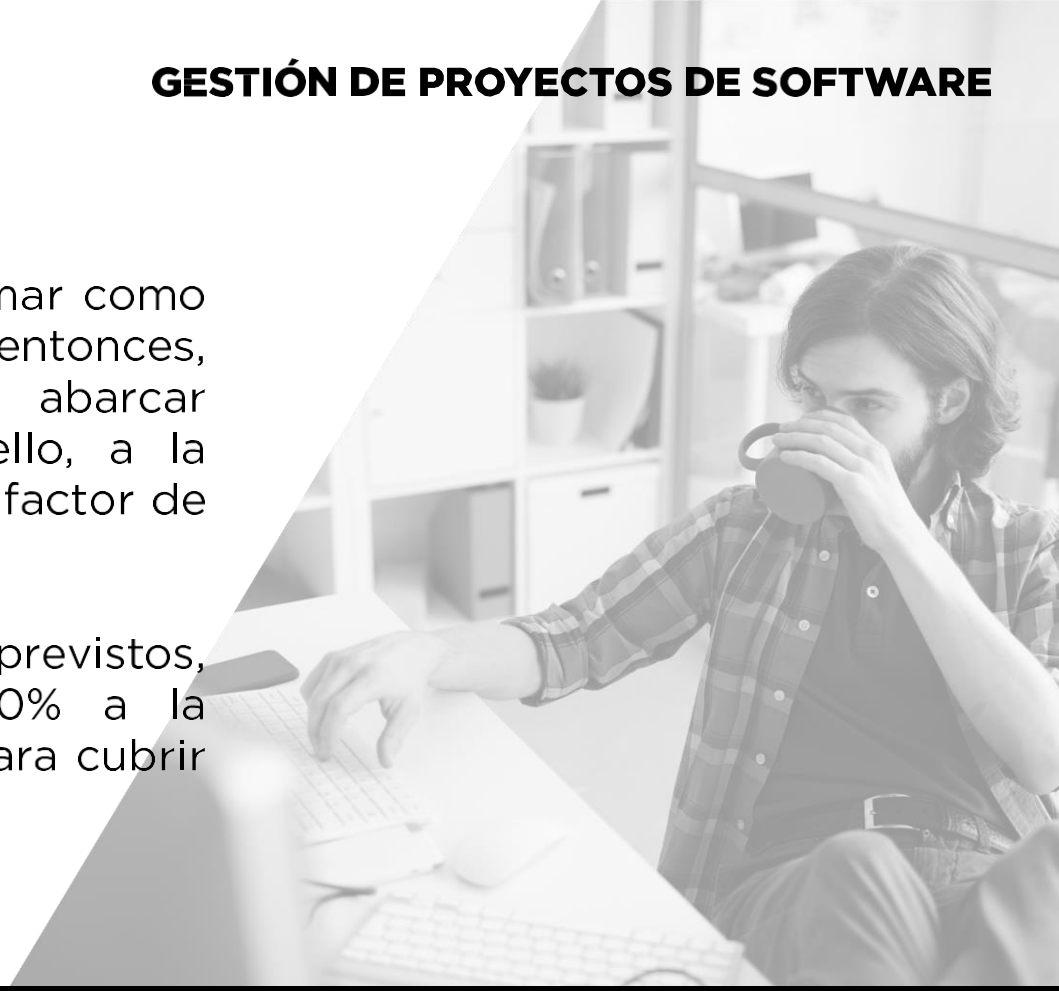
• Deben evitarse situaciones en las que el proyecto entero se retrase debido a que no se ha terminado una “actividad crítica”.

• Las actividades del proyecto deben durar por lo menos una semana. Hacer subdivisiones más finas significa invertir tiempo innecesario en la estimación y revisión de tablas.

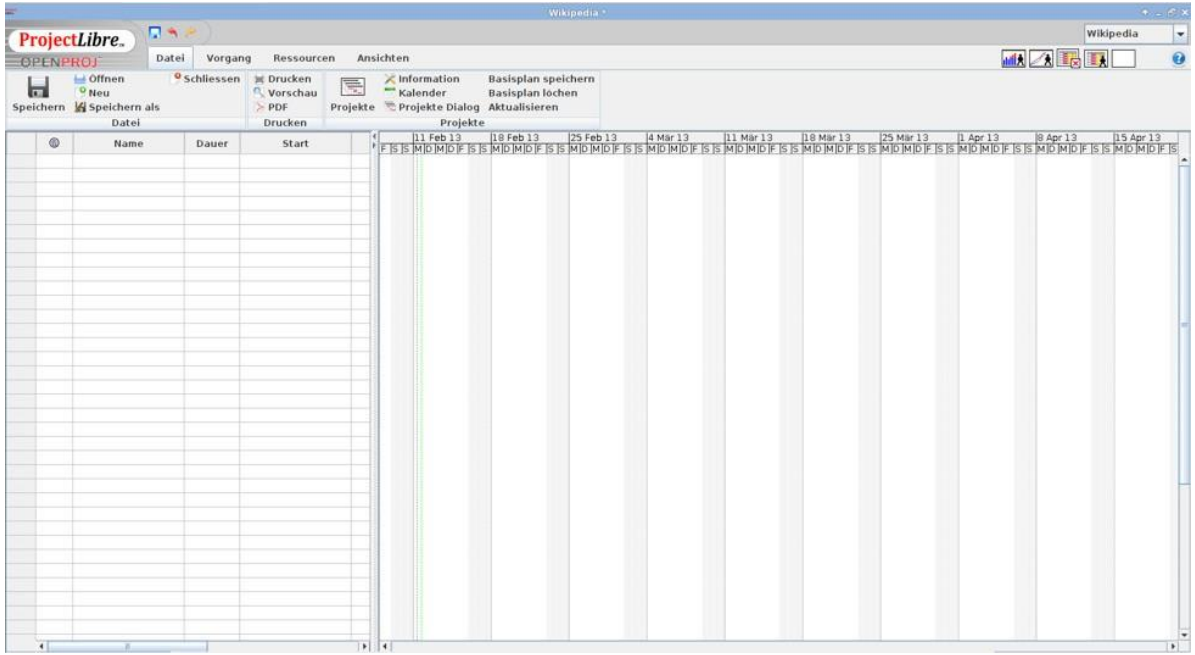
• Es útil asignar máximo de 8 a 10 semanas para realizar cualquier actividad. Si requieren más tiempo, se deben dividir.

Una buena regla práctica es estimar como si nada fuera a salir mal, y, entonces, incrementar la estimación para abarcar los problemas previstos. Para ello, a la estimación se le debe agregar un factor de contingencia adicional.

Como regla, para los problemas previstos, siempre debe agregarse un 30% a la estimación original, y otro 20% para cubrir algunas cosas no previstas.



Las herramientas de gestión de proyectos, como **Microsoft Project o Project Libre**, se utilizan para automatizar la producción de diagramas.



Los diagramas de Gantt y las redes de actividades son notaciones gráficas que ilustran la calendarización del proyecto:

- **Diagramas de Gantt.** Muestran quién es responsable de cada actividad, así como las fechas de inicio y fin.

- **Redes de actividades.** Muestran las dependencias entre las actividades que conforman un proyecto.



El tiempo mínimo requerido para finalizar el proyecto se estima teniendo en cuenta la trayectoria más larga en la red de actividades (el camino crítico).

El **camino crítico** se muestra como una sucesión de rectángulos más resaltados. Cualquier aplazamiento al completar una actividad crítica provoca retraso en el proyecto, ya que las actividades siguientes no pueden comenzar hasta que se finalice la actividad retrasada.

Sin embargo, los retrasos de las actividades que no están ligadas al camino crítico no provocan necesariamente un aplazamiento en todo el calendario. Mientras los retrasos no se propaguen a estas actividades, como para que se exceda el tiempo total del camino crítico, el proyecto no se verá afectado.

El **Método de la Ruta Crítica (CPM)** y la **Técnica de Revisión y Evaluación de Programas (PERT)** son ejemplos de cómo aplicar los planteamientos anteriores.

Finalmente, se deben calcular los **costos del proyecto.** Existen **tres parámetros** involucrados en el cálculo del costo de un proyecto de software:

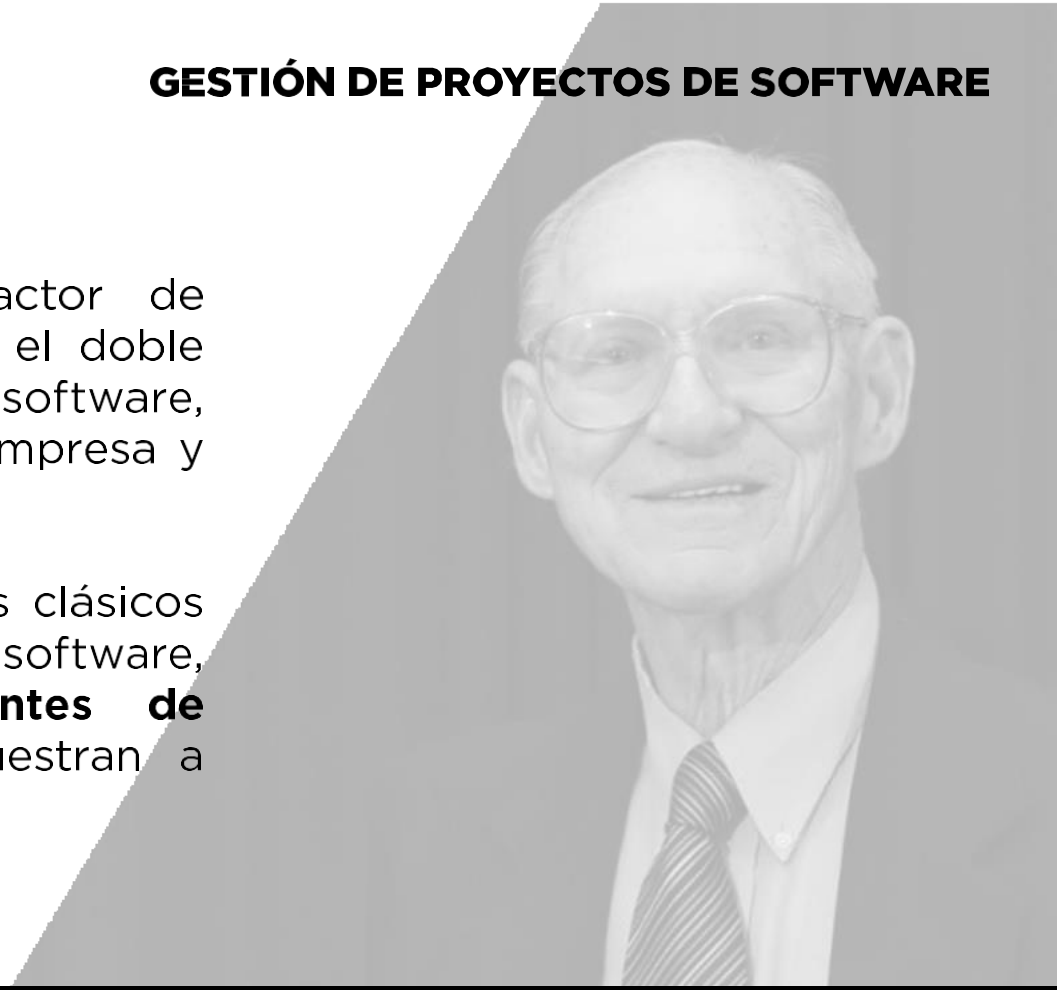
- Hardware y software, incluyendo el mantenimiento.
- Viajes y capacitación.
- Mano de obra.

Por tanto, **los siguientes costos deben incluirse:**

- Costos de proveer, aclimatar e iluminar las oficinas.
- Los costos del personal de apoyo como administrativos, secretarias, limpiadores y técnicos.
- Los costos de redes y las comunicaciones.
- Los costos de los recursos centralizados como las bibliotecas, los recursos recreativos, etc.
- Los costos de seguridad social, pensiones, seguros privados, etc.

En este sentido, existe un factor de sobrecarga que normalmente es el doble del salario de un ingeniero de software, dependiendo del tamaño de la empresa y sus sobrecargas asociadas.

Para Barry W. Boehm, uno de los clásicos en ingeniería de construcción de software, existen **siete métodos diferentes de estimación.** Mismos que se muestran a continuación:



- La utilización de modelos algorítmicos que dan una estimación del costo en función de un número determinado de variables que influyen en el costo.

- El juicio experto de profesionales que han trabajado en proyectos similares.

- La analogía con otro proyecto parecido que sea comparable con el que se plantea.

- La utilización de los recursos disponibles. Aquí, lo que limita el costo es el volumen de recursos de los que se dispone en cada etapa.

• El precio ganador, en el que la estimación de los costos no se realiza en función de las cargas de trabajo, sino de las condiciones del mercado y la competencia.

• La estimación global descendente, en la que se intenta fijar un costo general del proyecto a partir de sus características principales como el tamaño, complejidad, dificultad técnica, calidad y fiabilidad, entre muchas otras características.

• La estimación ascendente a partir de la desagregación de las actividades en tareas.

VIDEO

Te invitamos a ver el siguiente video:

Planificación del proyecto de software



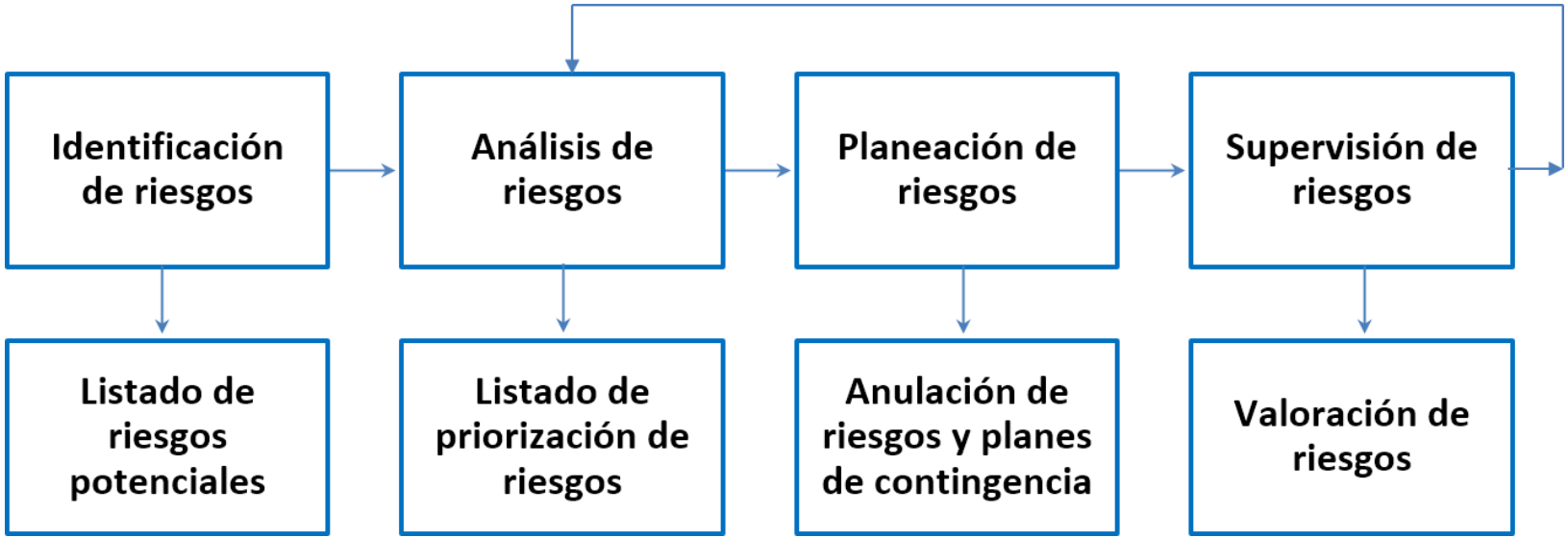
GESTIÓN DEL RIESGO EN PROYECTOS DE SOFTWARE

Una tarea importante del gestor de proyectos es **anticipar los riesgos** que podrían afectar a la programación del proyecto o a la calidad del software a desarrollar, así como emprender acciones para evitarlos.

Los resultados de este análisis de riesgos se deben documentar a lo largo del plan del proyecto junto con el análisis de consecuencias cuando el riesgo ocurra. Identificar estos y crear planes para minimizar sus efectos en el proyecto se llama **gestión de riesgos.**



Es preciso anticiparse a los riesgos y tener planes de contingencia para que sea posible aplicar **acciones de recuperación.** El proceso de gestión de riesgos se presenta a continuación:



Etapas del proceso de gestión de riesgos:

- **Identificación de riesgos.** Identificar los posibles riesgos para el proyecto, el producto y los negocios.

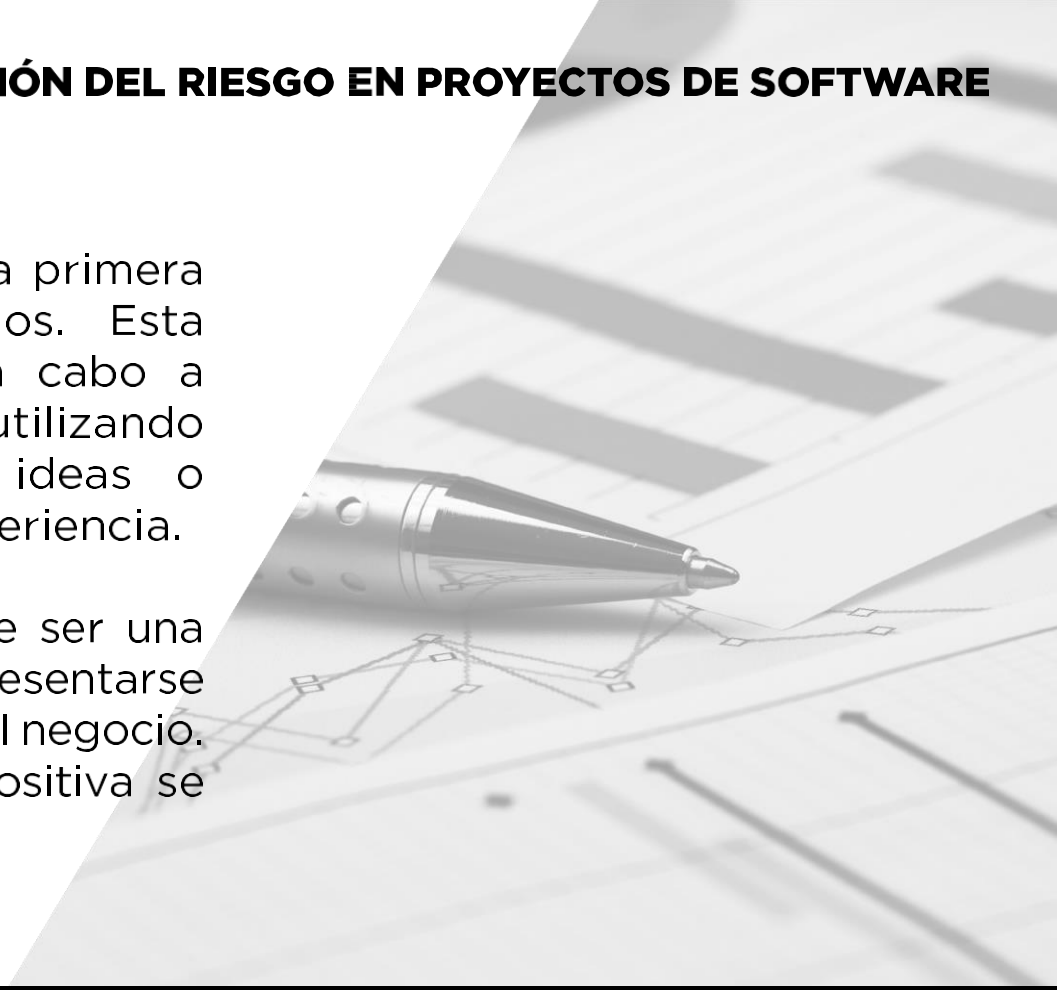
- **Análisis de riesgos.** Valorar las probabilidades y consecuencias de estos riesgos.

- **Planificación de riesgos.** Crear planes para abordar los riesgos, ya sea para evitarlos o minimizar sus efectos en el proyecto.

- **Supervisión de riesgos.** Valorar los riesgos de forma constante y revisar los planes para su mitigación, tan pronto como la información de estos esté disponible.

La **identificación de riesgos** es la primera etapa de la gestión de riesgos. Esta identificación se puede llevar a cabo a través de un proceso de grupo, utilizando un enfoque de tormenta de ideas o simplemente basándose en la experiencia.

El resultado de este proceso debe ser una larga lista de riesgos que podrían presentarse y afectar al producto, al proceso o al negocio. Por ejemplo, en la siguiente diapositiva se listan **algunos riesgos comunes:**



CATEGORÍAS	RIESGOS
TECNOLOGÍA	La base de datos que se utiliza en el sistema no puede procesar muchas transacciones por segundo como se esperaba. Los componentes de software que deben reutilizarse contienen defectos que limitan su funcionalidad.
PERSONAL	Es imposible reclutar personal con las habilidades requeridas para el proyecto. El personal clave está enfermo y no disponible en momentos críticos. La capacitación solicitada para el personal no está disponible.
ORGANIZACIONAL	La organización se reestructura de tal forma que una gestión diferente se responsabiliza del proyecto. Los problemas financieros de la organización fuerzan a reducciones en el presupuesto del proyecto.

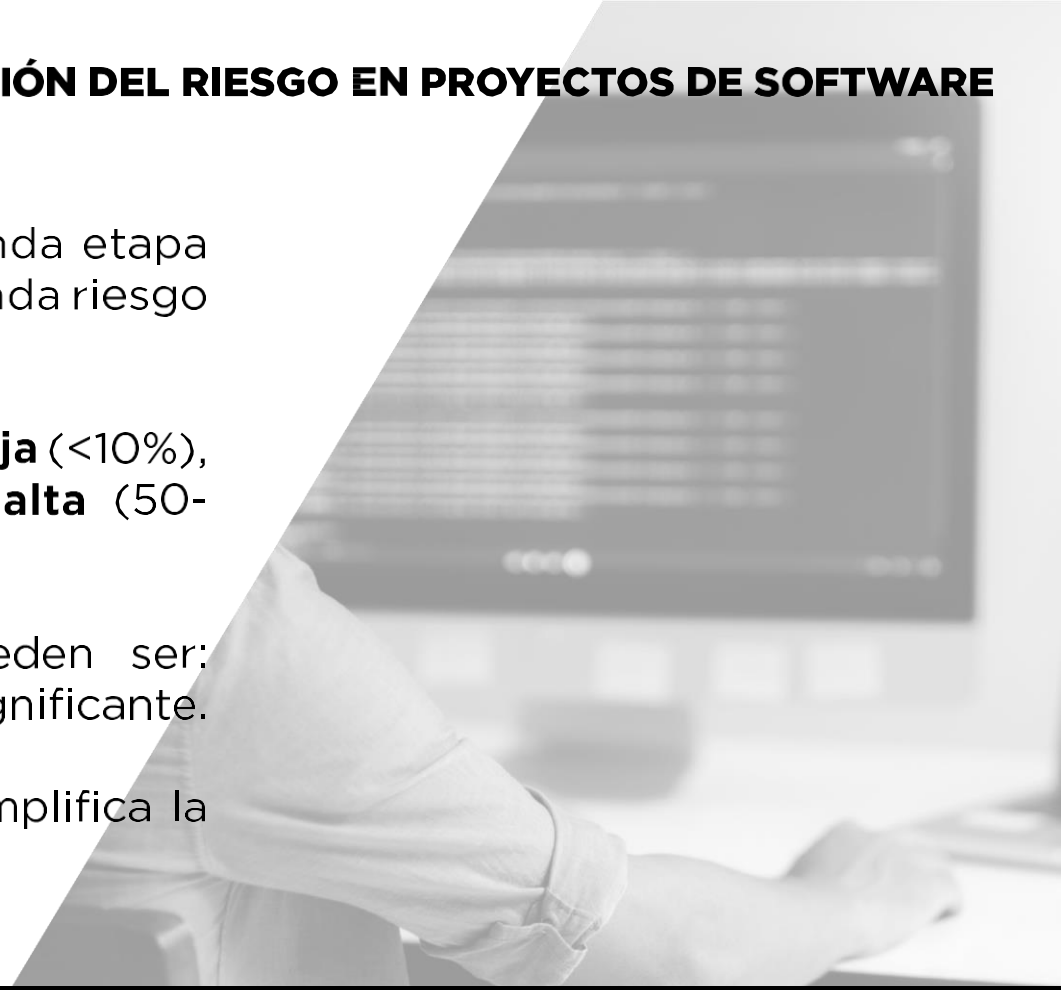
CATEGORÍAS	RIESGOS
HERRAMIENTAS	Es ineficiente el código generado por las herramientas CASE. Estas no se pueden integrar.
REQUERIMIENTOS	Se proponen cambios en los requerimientos que necesitan rehacer el diseño. Los clientes no comprenden el impacto de los cambios en los requerimientos.
ESTIMACIÓN	El tiempo requerido para desarrollar el software está subestimado. La tasa de reparación de defectos está subestimada. El tamaño del software está subestimado.

El **análisis de riesgos** es la segunda etapa de la gestión de riesgos y evalúa cada riesgo y su probabilidad:

- La probabilidad puede ser **muy baja** (<10%), **baja** (10-25%), **media** (25-50%), **alta** (50-75%) o **muy alta** (>75%).

- Los impactos del riesgo pueden ser: catastrófico, serio, tolerable o insignificante.

En la siguiente diapositiva se ejemplifica la situación:



RIESGO	PROBABILIDAD	IMPACTO
Los problemas financieros de la organización fuerzan a reducir el presupuesto del proyecto.	*Baja*	*Catastrófico*
Es imposible reclutar personal con las habilidades requeridas para el proyecto.	*Alta*	*Catastrófico*
El personal clave está enfermo y no disponible en momentos críticos	*Moderada*	*Serio*
Los componentes de software que deben reutilizarse contienen defectos que limitan su funcionalidad.	*Moderada*	*Serio*
Se proponen cambios en los requerimientos que necesitan rehacer el diseño.	*Moderada*	*Serio*

RIESGO	PROBABILIDAD	IMPACTO
La organización se reestructura de tal forma que cambia el grupo de gestión.	*Alta*	*Serio*
La base de datos que se utiliza en el sistema no puede procesar muchas transacciones por segundo como se esperaba	*Moderada*	*Serio*
El tiempo requerido para desarrollar el software está subestimado.	*Alta*	*Serio*
Las herramientas CASE no se pueden integrar.	*Alta*	*Tolerable*

La **planificación de riesgos,** por su parte, se enfoca en las estrategias para gestionar los riesgos identificados. Estas pueden dividirse en tres categorías:

- **Estrategias de prevención.** Sirven para reducir la ocurrencia.

- **Estrategias de minimización.** Sirven para reducir el impacto.

- **Planes de contingencia.** Sirven para prepararse para lo peor.

En la siguiente diapositiva se ejemplifica la situación:



RIESGOS	ESTRATEGIA
PROBLEMAS FINANCIEROS DE LA ORGANIZACIÓN	Preparar un documento breve para el gestor principal que muestre que el proyecto hace contribuciones muy importantes a las metas del negocio
PROBLEMAS DE RECLUTAMIENTO	Alertar al cliente de las dificultades potenciales y los posibles retrasos, así como investigar la compra de componentes.
ENFERMEDAD DE PERSONAL	Reorganizar el equipo de tal forma que haya solapamiento en el trabajo y las personas comprendan el de los demás.

RIESGOS	ESTRATEGIA
COMPONENTES DEFECTUOSOS	Reemplazarlos por los adquiridos con proveedores conocidos y fiables.
CAMBIOS DE LOS REQUERIMIENTOS	Rastrear la información para valorar el impacto de los requerimientos, maximizar la información oculta en ellos.
REESTRUCTURACIÓN ORGANIZACIONAL	Preparar un documento breve para el gestor principal, el cual debe mostrar que el proyecto hace contribuciones muy importantes a las metas del negocio.

Finalmente, la **supervisión de riesgos** valora cada riesgo para decidir si es más o menos probable, y si han cambiado sus efectos. En particular, da seguimiento a la probabilidad de ocurrencia y al impacto planificado.

En la siguiente diapositiva se ejemplifica la situación:



CATEGORÍAS	FACTOR DE RIESGO
TECNOLOGÍA	Entrega retrasada del hardware o de la ayuda del software, así como muchos problemas reportados.
PERSONAL	Baja moral del personal, así como malas relaciones entre los miembros del equipo y disponibilidad de empleo.
ORGANIZACIONAL	Chismorreo organizacional y falta de acciones por parte del gestor principal.

CATEGORÍAS	FACTOR DE RIESGO
HERRAMIENTAS	Rechazo de los miembros del equipo para utilizar herramientas, quejas acerca de las herramientas CASE, así como peticiones de estaciones de trabajo más potentes.
REQUERIMIENTOS	Peticiones de muchos cambios en los requerimientos, así como quejas del cliente.
ESTIMACIÓN	Fracaso en el cumplimiento de los tiempos acordado, así como en la eliminación de defectos reportados.

LECTURA

Gestión de riesgos en proyectos de software educativo



Actividad 2 – Unidad 3. Control de Versiones de Software

Te invitamos a realizar la siguiente actividad:

Presiona el botón para descargar la actividad



Presiona el botón para entregar la actividad



CONCLUSIÓN

Los administradores o gestores de software realizan principalmente las siguientes actividades: redacción de la propuesta, planificación y calendarización del proyecto, así como la estimación de costos, supervisión y revisión del proyecto; selección y evaluación del personal; redacción y presentación de informes; análisis y gestión del riesgo, entre otras.

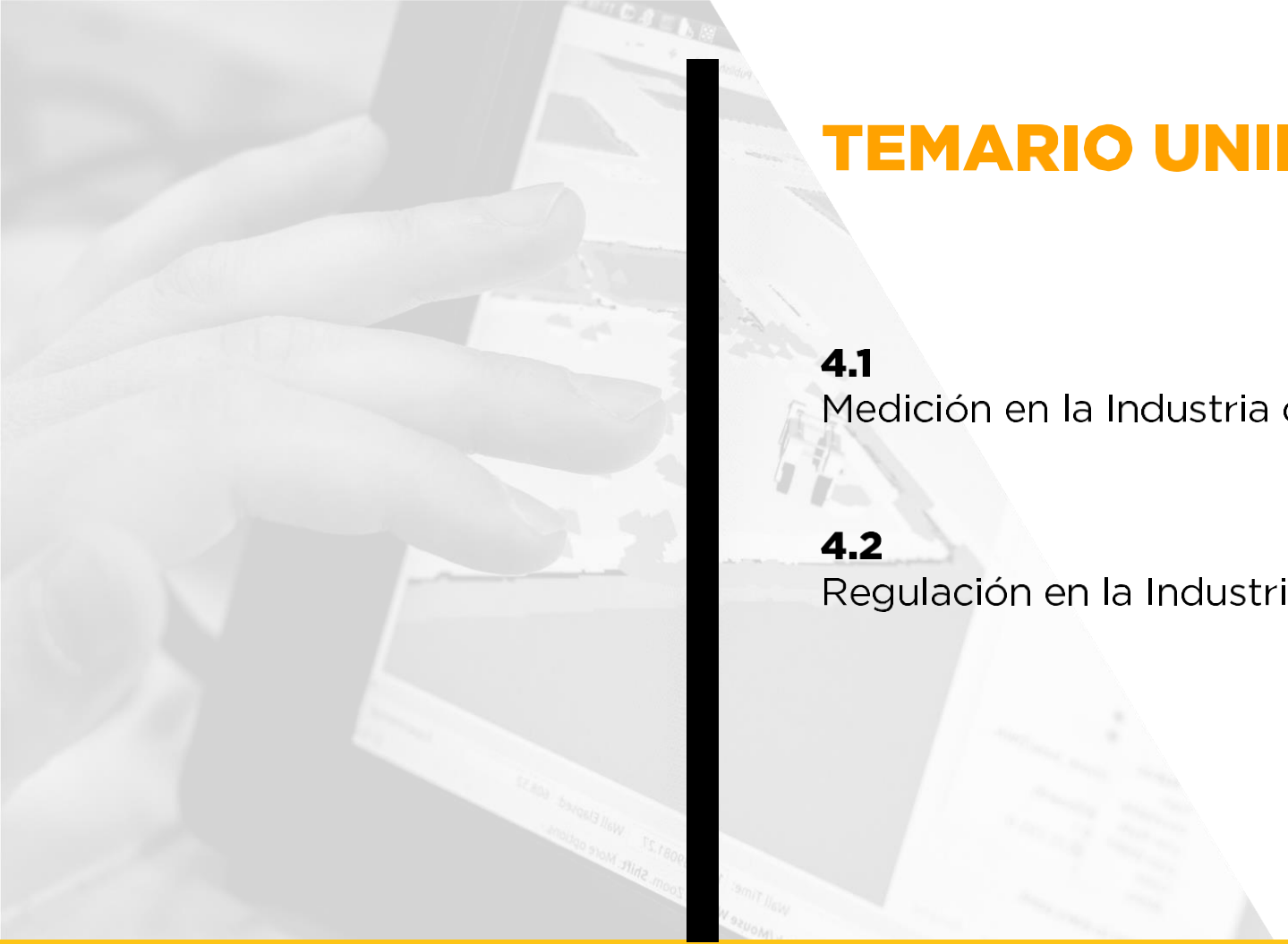
Un administrador de proyectos de software supervisa el trabajo para asegurar que este se lleva a cabo conforme a los estándares requeridos. De la misma manera, supervisan el progreso para comprobar que el desarrollo se ajusta al tiempo previsto y al presupuesto. Una tarea importante del gestor de proyectos es anticipar los riesgos que podrían afectar a la programación del proyecto o a la calidad del software a desarrollar, así como emprender acciones para evitar esos riesgos.

¡Felicidades!

Acabas de concluir la **tercera unidad** de tu curso **Ingeniería de Software I.** Te invitamos a finaliz r este esfuerzo realizando el examen parcial correspondiente.

Para ello, debes regresar a la pantalla principal y dar clic en *Presentar examen*.

UNIDAD 4
CIERRE DEL PROYECTO DE SOFTWARE



TEMARIO UNIDAD 4

4.1
Medición en la Industria del Software

4.2
Regulación en la Industria del Software

INTRODUCCIÓN

En esta cuarta unidad se explica por qué es importante medir el proceso de Ingeniería de Software. Lo anterior, para generar una mejora continua en los procesos internos y en los productos que genera la fábrica de software.

Finalmente, analizaremos cómo el derecho informático aplicado al software puede ayudarnos a evitar su piratería y uso ilegal.

COMPETENCIAS A DESARROLLAR

El alumno comprenderá:

• El concepto de métrica y sus distintos tipos.

El alumno conocerá:

• Algunos conceptos básicos del derecho informático aplicado al software



MEDICIÓN EN LA INDUSTRIA DEL SOFTWARE

La medición del software se refiere al uso de un valor numérico desde algún atributo del software o de su proceso. Puede perseguir **dos objetivos:**

- **Hacer predicciones generales acerca del sistema.** Haciendo mediciones de las características de los componentes del sistema y reuniéndolas, podremos derivar una estimación general de algunos atributos del sistema, como el número de fallas.

- **Identificar componentes anómalos.** Mediante las mediciones, se pueden identificar los componentes que salgan de lo normal.

Una **métrica de software** es cualquier tipo de medición relacionada con un sistema, proceso o documentación de software. Son medidas de composición simple.

Entendemos por **indicador** aquel valor de una variable que anticipe el valor de la medida de un resultado. Es más sofisticado que las métricas, ya que relaciona dos o más variables.

Cada indicador debe cumplir con la siguiente información:

- ¿Quién lo mide? (en algunos casos es **quien lo revisa o valida**)

- ¿Cómo se mide o evalúa? **(forma de cálculo)**





- ¿Quién es el usuario del dato? **(forma de presentación)**

- ¿Para qué se mide? **(uso)**

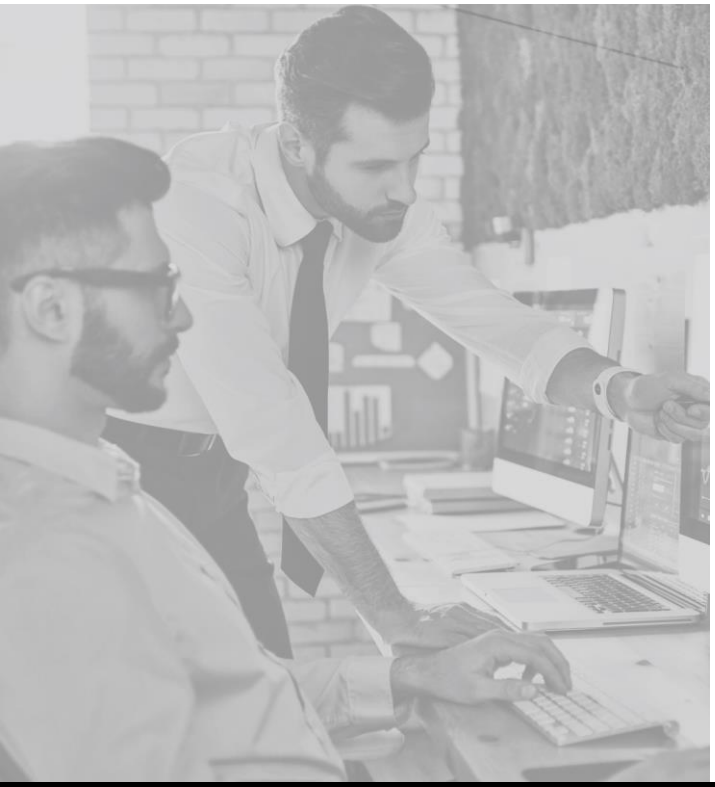
- ¿Cuál es el valor objetivo del indicador? **(rangos aceptables e inaceptables)**



MEDICIÓN EN LA INDUSTRIA DEL SOFTWARE

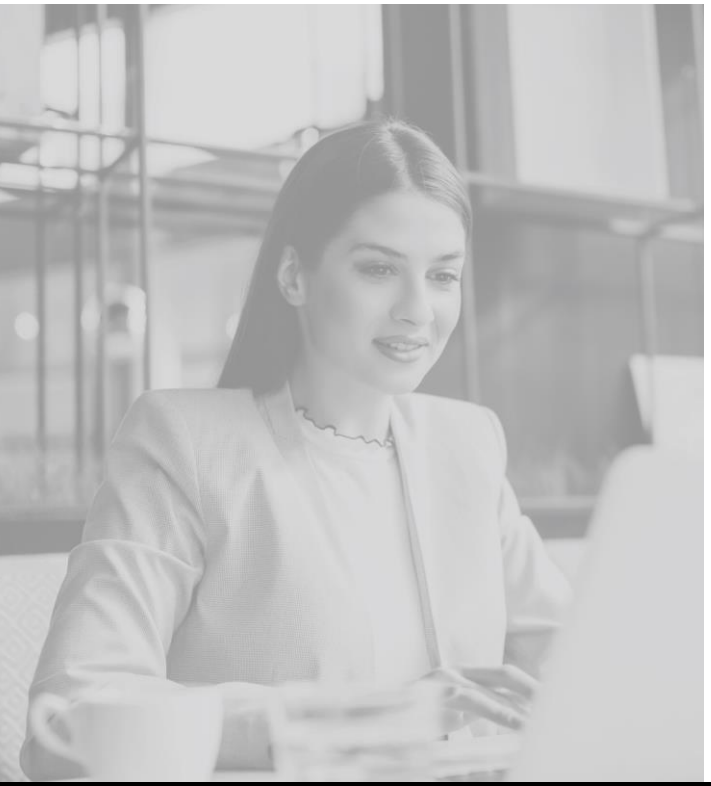
Las **métricas**, generalmente, se clasifican en factores como los siguientes:

- **Humano**
- **Económico**
- **Funcional**
- **Productivo**
- **Técnico**
- **Operativo**
- **Temporal**
- **Historial**



El **factor humano** introduce dos tipos de medidas:

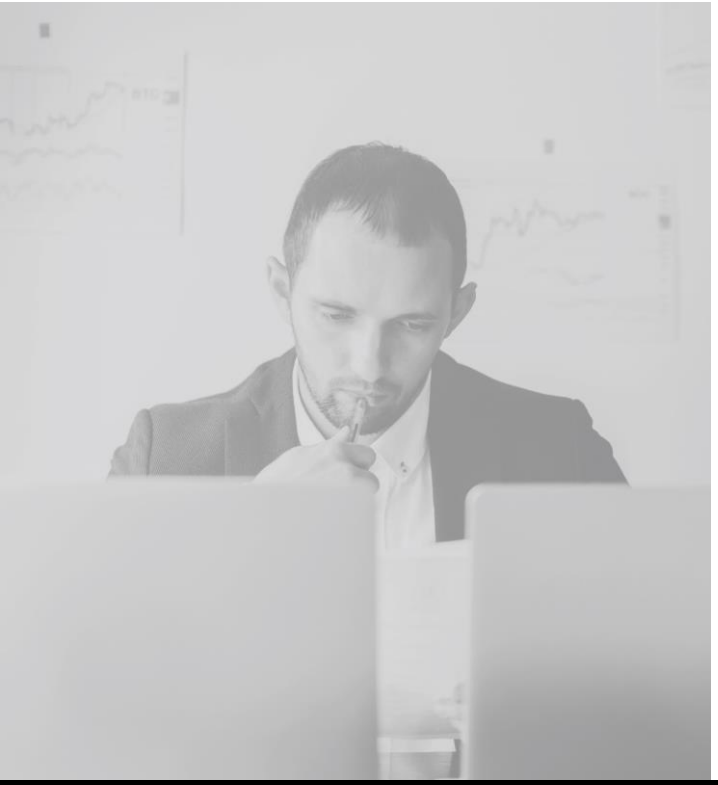
• **Horas hombre.** Al medir un proyecto, la cantidad de horas hombre representa el tiempo que este tarda en realizar dicho trabajo. Esta no es una medida lineal, por lo que si se colocan dos hombres a realizar el proyecto, no necesariamente se divide el tiempo a la mitad. Tampoco es una medida exacta, debido a que su error crece al mismo tiempo que lo hace la unidad de tiempo.



- **Apreciación de usuarios.** Ha sido un error muy frecuente no tomar en cuenta el parecer de los usuarios al adquirir o desarrollar un producto de software. El usuario es el que al final utiliza el producto y quien mejor conoce sus capacidades. Uno de los primeros puntos por enfocar es el alcance del software. Los usuarios, como ya se mencionó, son las personas que conocen las necesidades actuales y lo que el software debe cumplir para satisfacerlas.



El **factor económico,** por otro lado, introduce medidas que no necesariamente son monetarias. Distintas medidas ayudan a realizar comparaciones sobre la economía del equipo. El factor económico se basa en la comparación de costo-beneficio. De esta manera se toman decisiones sobre la viabilidad de los proyectos. Las métricas monetarias que se aplican en el software directamente pueden ser: el precio del producto o su mantenimiento.

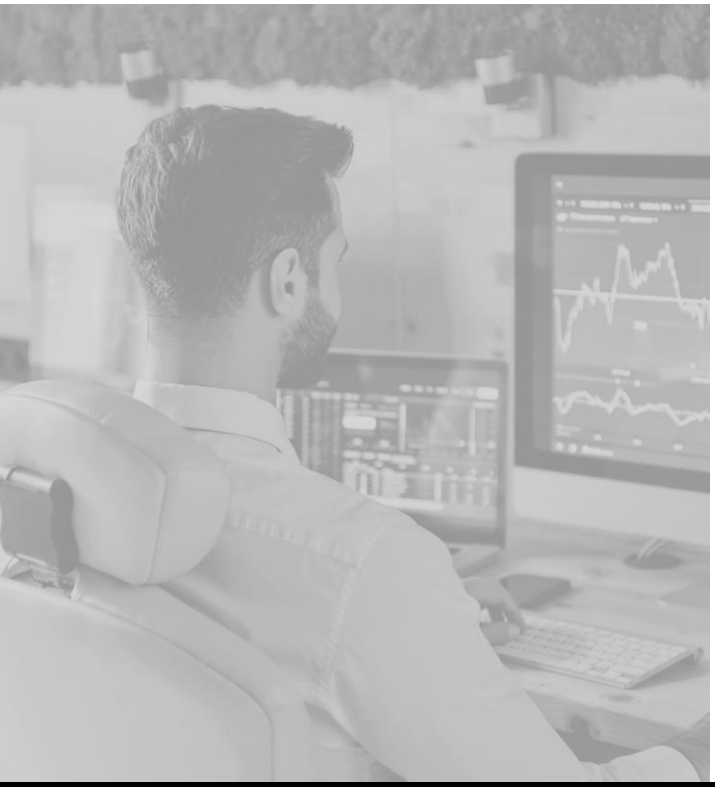


El **factor funcional** introduce las unidades de eficiencia. Estas representan el grado en que un producto genera lo que se espera de él.

El **factor productivo** trabaja con medidas de producción. Estas son utilizadas para la determinación del grado de beneficios. En general, representan la cantidad o calidad de lo que se produce directamente con el resultado del proyecto.



El **factor técnico** se encarga de las medidas que representan el costo de la implementación y el mantenimiento. Estas medidas no son monetarias, sino técnicas. Para la puesta en marcha, se mide con puntos el efecto sobre la conectividad, el reajuste de estructura, la migración de datos, la introducción de datos, la capacitación de usuarios y técnicos, etcétera.



El **factor operativo** es el encargado de medir el manejo del producto. Ciertas medidas que deben ser tomadas en cuenta son: tiempo-hombre utilizado en la operación diaria, cantidad y complejidad de procesos operativos.

El **factor temporal** trabaja en forma estadística sobre: los tiempos de procesos, los tiempos de mantenimiento, los tiempos de respuesta y los tiempos de inicialización.

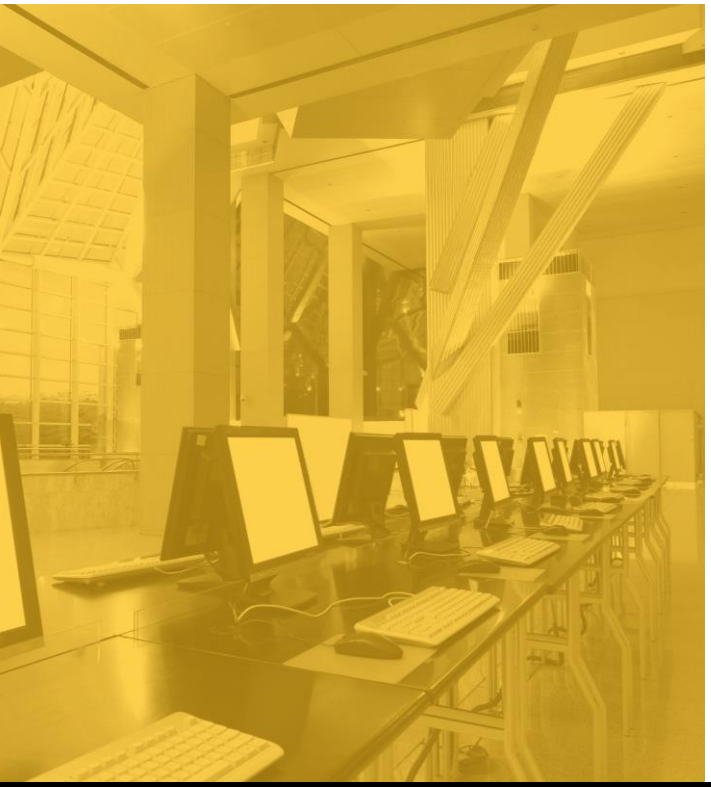


El **factor historial** se basa en la historia del software. Las medidas son completamente heurísticas. Es decir, si un software está siendo utilizado por más de una compañía, y lo ha estado haciendo por mucho tiempo, debe ser bueno. Para especificar la medida, se requiere tener un historial muy completo del software, que al menos contemple los siguientes datos:

- Fecha de salida al mercado
- Plataformas en las que corre
- Nombre de la compañía creadora



- Nombre de la compañía distribuidora
- Si ha sido cambiado de compañía (comprado)
- Fallos importantes registrados
- Comentarios de terceros
- Índices de ventas
- Comparación en revistas
- Personal que lo ha trabajado y que esté en la compañía



- Tiempo y versiones que han salido al mercado
- Año de la última versión
- Características cambiadas a través de los años
- Recursos que se utilizan de la computadora y periféricos

REGULACIÓN EN LA INDUSTRIA DEL SOFTWARE

Uno de los temas regulatorios más importantes en la industria del software ha sido la **piratería.** Esta consiste en la distribución, copia o uso ilegal de software sujeto a la propiedad de derechos de autor.

Los piratas de software renuncian al derecho de recibir actualizaciones y soporte técnico, pero obtienen el uso del programa sin pagar por él.

La piratería es un problema enorme debido a que es muy fácil de hacer.

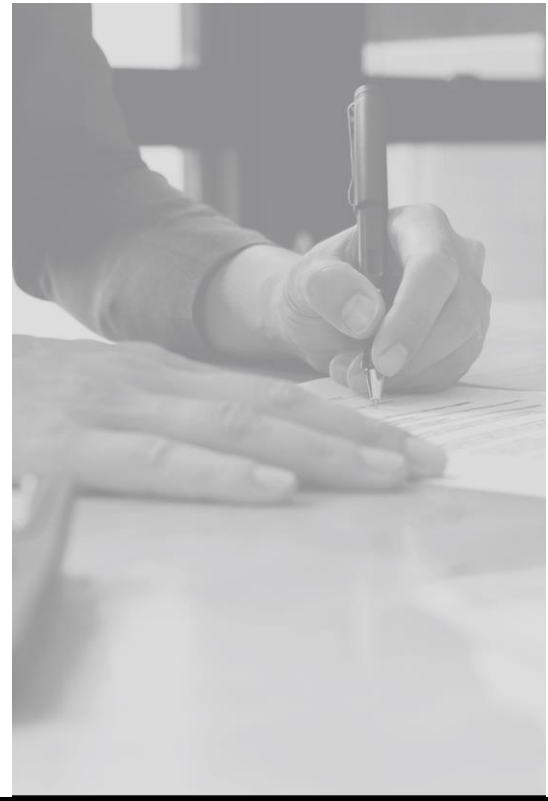
Hasta hace poco tiempo, la piratería de software, aunque aborrecida, era tolerada por algunos editores de software que consideraban impracticable hacer cumplir la ley de derechos de autor.

Últimamente, sin embargo, los editores más importantes de software han tomado una posición más agresiva, entablando demandas en contra de empresas que hacen copias ilegales.

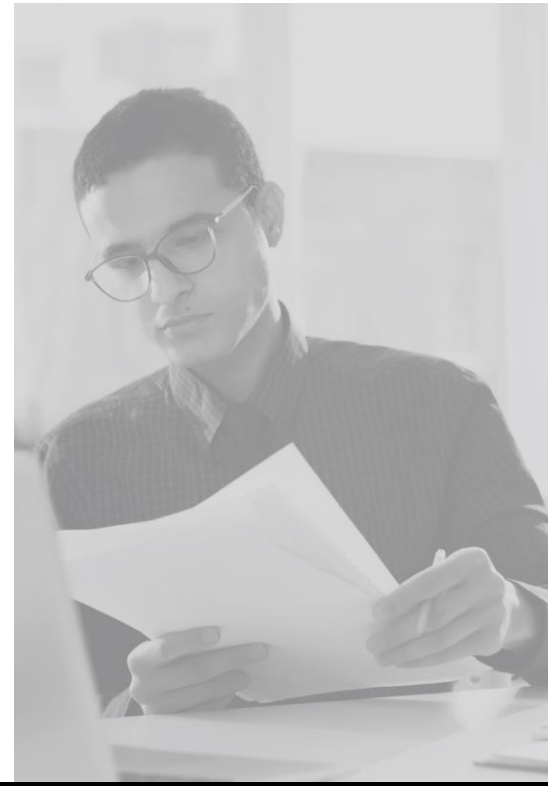
Además de las sanciones civiles, la ley de derecho de autor impone multas y encarcelamientos como castigos de la infracción.

El **derecho de autor** es el derecho de prohibir que otras personas realicen copias ilícitas. Lo que mucha gente no sabe es que el derecho de autor incluye, no uno, sino cinco derechos exclusivos, que son los siguientes:

- El derecho exclusivo de hacer copias
- El derecho exclusivo de distribuir copias al público
- El derecho exclusivo de preparar obras derivadas
- El derecho exclusivo de representar la obra en público (en el caso de teatro, danza, etc.)
- El derecho exclusivo de exhibir la obra en público (en el caso de cintas cinematográficas)



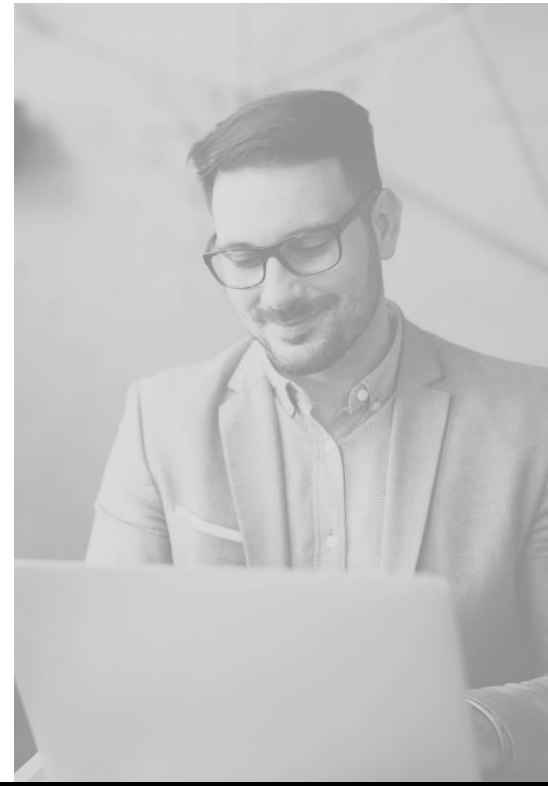
En México, los derechos de autor del software se registran ante el INDAUTOR (Instituto Nacional del Derecho de Autor). Puedes consultar los documentos requeridos, formatos y costos dando clic en el siguiente botón:



En ningún momento, un usuario que compra un programa, se convierte en propietario de este. Solo adquiere su derecho de uso.

Las **licencias** son contratos suscritos entre los productores de software y los usuarios.

Existen distintos tipos de licencias de software privativas y de uso libre. Algunos ejemplos son los siguientes:



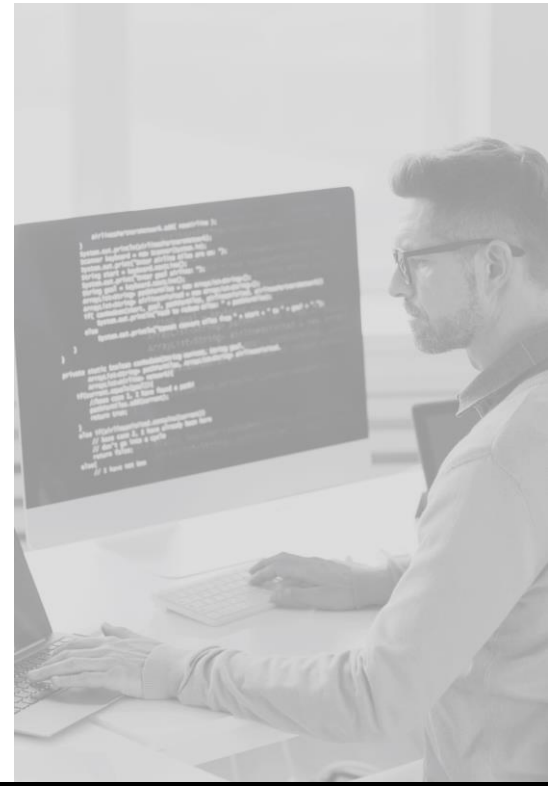
Software propietario:

- Software cerrado, donde el dueño controla su desarrollo y no divulga sus especificaciones.
- Es el producido principalmente por las grandes empresas.
- Antes de utilizarlo se debe pagar por su licencia.
- Normalmente se tiene derecho a utilizarlo en una sola computadora, un solo usuario y a realizar una copia de respaldo, pero la redistribución o copia para otros propósitos no es permitida.



Software de demostración:

- Comúnmente llamado “software demo”.
- Son programas que no son 100 % funcionales, o dejan de trabajar al cabo de cierto tiempo.
- Suelen conseguirse en Internet o en los kioscos de periódicos y revistas.
- Similar al software *shareware,* por la forma en que se distribuye.



Software de dominio público:

Es software libre que tiene como particularidad la ausencia de las restricciones del *copyright.* Es decir, garantiza las cuatro libertades esenciales del software, las cuales son las siguientes:

- La libertad de ejecutar el programa como se desee, con cualquier propósito **(libertad 0).**

- La libertad de estudiar cómo funciona el programa, y cambiarlo para que haga lo que usted quiera **(libertad 1).**



• La libertad de redistribuir copias para ayudar a otros **(libertad 2).**

•La libertad de distribuir copias de sus versiones modificadas a terceros **(libertad 3).**



Software semi-libre:

Para la *Free Software Foundation*, es software que posee las libertades del software libre, pero solo se puede usar para fines sin ánimo de lucro.



LECTURAS PARA REFORZAR EL CONTENIDO

• Peña Ayala, Alejandro. (2006). *Ingeniería de Software: Una guía para crear sistemas de información.* México. Primera Edición. IPN.

• *¿Qué es el software libre?* Accedido el 23 de enero 2021 en: https://www.gnu.org/philosophy/free-sw.html

CONCLUSIÓN

Las métricas del software son utilizadas para mejorar los sistemas, el proceso de desarrollo o la documentación. Los indicadores ayudan a identificar tendencias, corregir oportunamente desviaciones y dar seguimiento al cumplimiento de objetivos. En el cierre del proyecto se verifican las métricas correspondientes y se prepara el software para su comercialización.

La piratería, por su parte, consiste en la distribución, copia o uso ilegal de software sujeto a la propiedad de derechos de autor. Las sanciones civiles y la ley de derecho de autor imponen multas y encarcelamientos como castigos de la infracción a quienes realizan piratería. En México, los derechos de autor del software se registran ante el INDAUTOR.

¡Felicidades!

Acabas de concluir la **cuarta unidad** de tu curso **Ingeniería de Software I.** Te invitamos a finalizar este esfuerzo realizando el examen parcial correspondiente.

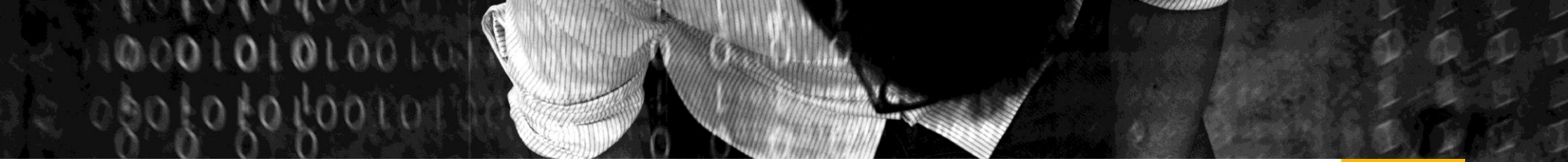
Para ello, debes regresar a la pantalla principal y dar clic en *Presentar examen*.

BIBLIOGRAFÍA

1. Sommerville, Ian. (2005). *Ingeniería del Software.* España. Séptima edición. Pearson Educación, S.A. ISBN: 84-7829-074-5

2. Braude. (2003). *Ingeniería del Software.* México. Primera edición. Ed. Alfa-Omega.

3. Pressman, Roger S. (2005). *Ingeniería del Software.* México. Sexta edición. Ed. McGraw-Hill.



PROYECTO FINAL



Proyecto Final. Evaluación de Desempeño

Te invitamos a realizar la siguiente actividad:

Presiona el botón para descargar la actividad



Presiona el botón para entregar la actividad

